



# Reporte Técnico de Actividades Práctico-Experimentales Nro. 2

## 1. Datos de Identificación del Estudiante y la Práctica

|                                              |                                                                                                                                                                                   |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Nombre de los estudiante(s)</b>           | Tyrone Encarnación<br>Erick Andrade<br>Jhoan Rojas<br>Matías Ortega                                                                                                               |
| <b>Asignatura</b>                            | Matemáticas Discretas                                                                                                                                                             |
| <b>Ciclo</b>                                 | 1A                                                                                                                                                                                |
| <b>Unidad</b>                                | Unidad 1: Lógica Matemática                                                                                                                                                       |
| <b>Resultado de aprendizaje de la unidad</b> | Desarrolla habilidades de razonamiento lógico para solucionar problemas ligados a la ingeniería, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| <b>Práctica Nro.</b>                         | 002 FASE 5-6                                                                                                                                                                      |
| <b>Título de la Práctica</b>                 | Operaciones Booleanas, Simplificación de Expresiones, Mapas de Karnaugh y Propiedades del Álgebra Booleana                                                                        |
| <b>Nombre del Docente</b>                    | Ing. Mario Enrique Cueva Hurtado                                                                                                                                                  |
| <b>Fecha</b>                                 | Semana 9 y 10: del 1 al 12 de junio                                                                                                                                               |
| <b>Horario</b>                               | Lunes, Martes, jueves 07:30 – 09:30                                                                                                                                               |
| <b>Lugar</b>                                 | Aula de la Carrera de Computación / Aula Virtual (Zoom)                                                                                                                           |
| <b>Tiempo planificado en el Sílabo</b>       | 18 horas (3 horas por sesión APE)                                                                                                                                                 |



## 2. Objetivo(s) de la Práctica

- Comprender los principios fundamentales del álgebra de Boole y su relación con la lógica proposicional estudiada en la Unidad 1.
- Aplicar las operaciones booleanas básicas (AND, OR, NOT, XOR, XNOR, NAND, NOR) en la construcción y simplificación de expresiones booleanas.
- Dominar las técnicas de simplificación de expresiones booleanas: algebraica, por mapas de Karnaugh (2, 3 y 4 variables), y propiedades del álgebra booleana.
- Diseñar funciones lógicas a partir de especificaciones dadas, expresándolas en diferentes formas (suma de productos, producto de sumas) y minimizándolas.
- Relacionar el álgebra de Boole con aplicaciones prácticas en computación: circuitos lógicos, diseño digital, consultas SQL y programación.

## 3. Materiales, Reactivos, Equipos y Herramientas

- Pizarra y marcadores
- Proyector multimedia
- Hojas de trabajo impresas con ejercicios de lógica proposicional y tablas de verdad
- Guía de ejercicios de inferencia y deducción lógica
- Calculadora científica
- Material bibliográfico de apoyo (textos digitalizados de referencia)
- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL para entrega de evidencias
- Entorno virtual de aprendizaje (EVA-UNL)
- Herramientas colaborativas: Google Drive, Wiki
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

#### **4. Procedimiento / Metodología Ejecutada**

##### **FASE 5 : Propiedades Adicionales del Álgebra de Boole (Semana 11: 15-19 junio 2026)**

- **Paso 1:** El docente presenta propiedades avanzadas del álgebra de Boole: teorema del consenso dual, funciones booleanas completas (NAND/NOR como puertas universales), expansión de Shannon, y resolución booleana.

- **Paso 2:** Los estudiantes investigan y presentan de forma participativa las leyes y propiedades

adicionales mediante ejemplos prácticos, conectando cada propiedad con aplicaciones reales en diseño digital y programación.

- **Paso 3:** Práctica de implementación de funciones booleanas usando únicamente compuertas

NAND o NOR (puertas universales): los estudiantes diseñan circuitos equivalentes usando solo un tipo de compuerta.

- **Paso 4:** Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales, analizando cómo la simplificación booleana reduce costos, consumo

energético y huella ambiental en la fabricación de circuitos.

- **Paso 5:** Práctica integradora: cada equipo resuelve un caso aplicado a la computación (diseño de un sumador binario de 1 bit, un comparador digital, o un codificador/decodificador) aplicando todas las técnicas de simplificación aprendidas.

##### **FASE 6: Evaluación Sumativa y Portafolio Integrador (Semana 12: 22-26 junio 2026)**

- **Paso 1:** Los equipos entregan el portafolio integrador de la unidad que incluya: todos los

talleres de simplificación booleana, mapas de Karnaugh resueltos, diseño de funciones lógicas, y el informe reflexivo sobre optimización.

- **Paso 3:** Sesión de socialización: los equipos presentan el caso aplicado que diseñaron



(sumador, comparador, etc.), explicando el proceso de diseño, las simplificaciones realizadas

y las ventajas obtenidas.

- **Paso 4:** Coevaluación entre equipos mediante rúbricas. Retroalimentación del docente sobre

el portafolio y la presentación. Análisis de casos aplicados a la computación.

---

## 5. Resultados

**5.1 FASE 5: Los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos.**

### **GUÍA DE INVESTIGACIÓN Y PRESENTACIÓN: LEYES Y PROPIEDADES ADICIONALES DEL ÁLGEBRA BOOLEANA**

#### **1. TEOREMA DEL CONSENSO DUAL (DUAL CONSENSUS THEOREM)**

##### **Definición y Explicación**

Mientras que el teorema del consenso estándar opera sobre una suma de productos, el Teorema del Consenso Dual se aplica directamente a expresiones en forma de producto de sumas (POS). Este teorema establece que en una combinación de tres variables lógicas sumadas en parejas, uno de los términos de la multiplicación es completamente redundante y puede ser eliminado sin alterar el comportamiento o resultado de la función.

##### **Identidad Matemática**

$$(A + B) \cdot (A' + C) \cdot (B + C) = (A + B) \cdot (A' + C)$$

Donde A' significa "A negado".

##### **Demostración intuitiva**

Analicemos el término  $(B + C)$ . Si tanto B como C son verdaderos (1), este paréntesis es verdadero. Sin embargo, debido a que la variable A debe tomar obligatoriamente un valor binario (ya sea 1 o 0), su presencia en los dos primeros términos ( $(A + B)$  y  $(A' + C)$ ) forzará a que uno de ellos dependa exclusivamente de los valores de B o C. Esto hace que el tercer paréntesis sea una repetición lógica que no aporta nueva información.

##### **Ejercicio Práctico Resuelto**

**Enunciado:** Simplificar la siguiente expresión lógica utilizada para el control de un circuito de automatización:

$$F = (X + Y) \cdot (X' + Z) \cdot (Y + Z)$$

## Solución Paso a Paso

### Reglas y propiedades adicionales del álgebra booleana

Ejercicio Práctico Resuelto:

Simplificar la siguiente expresión lógica utilizada para el control de un circuito de automatización:

$$F = (X + Y) (\bar{X} + Z) (Y + Z)$$

Solución Paso a Paso:

1. Identificamos las variables correspondientes a la identidad del teorema

$$A = X, B = Y, C = Z$$

2. Buscamos el término de consenso redundante, el cual está formado por las variables que acompañan a la variable que cambia de estado ( $X$  y  $\bar{X}$ ). En este caso, las variables acompañantes son  $Y$  y  $Z$ .

3. El término de la expresión

4. Eliminamos el término de la expresión.

Resultado Simplificado:  $F = (X + Y) (\bar{X} + Z)$

### Resultado Simplificado:

$$F = (X + Y) \cdot (X' + Z)$$

### Aplicación en la Vida Real y Conexión con Diseño Digital

En el diseño de hardware y circuitos lógicos integrados, implementar la función original:

$$F = (X + Y) \cdot (X' + Z) \cdot (Y + Z)$$

requiere físicamente tres compuertas OR independientes que luego conectan sus salidas a una compuerta AND de tres entradas.

Al aplicar el Teorema del Consenso Dual, optimizamos el circuito reduciéndolo a sólo dos compuertas OR y una compuerta AND de dos entradas.

## 2. ÁLGEBRA BOOLEANA GENERAL: LEYES DE DE MORGAN

### Definición y Explicación

Las Leyes de De Morgan son herramientas fundamentales para la transformación de compuertas lógicas y la simplificación de condiciones complejas. Explican la equivalencia matemática que existe al distribuir o retirar una negación (operador NOT) de un paréntesis que contiene operaciones OR o AND.

### Identidades Matemáticas

#### Primera Ley:

$$(A \cdot B)' = A' + B'$$

(La multiplicación negada se convierte en suma de variables negadas)

#### Segunda Ley:

$$(A + B)' = A' \cdot B'$$

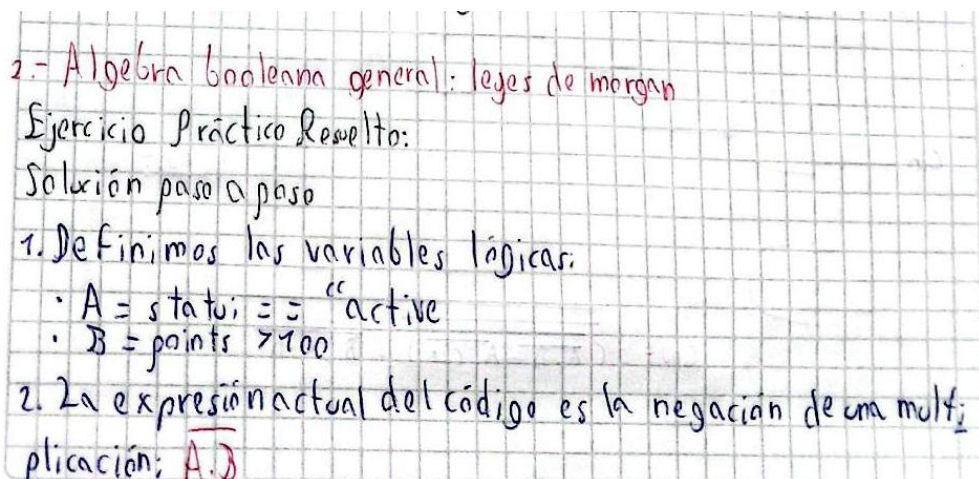
(La suma negada se convierte en multiplicación de variables negadas)

### Ejercicio Práctico Resuelto

**Enunciado:** Un sistema de desarrollo de software cuenta con la siguiente condición lógica anidada:

if not (status == "active" and points > 100):

### Solución Paso a Paso



2.- Álgebra booleana general: leyes de morgan

Ejercicio Práctico Resuelto:

Solución paso a paso

1. Definimos las variables lógicas:

- $A = \text{status} == \text{"active"}$
- $B = \text{points} > 100$

2. La expresión actual del código es la negación de una multiplicación:  $\overline{A \cdot B}$



3. Aplicamos la primera Ley de Morgan:  $\overline{A \cdot B} = \bar{A} + \bar{B}$ .

4. Negamos cada variable individualmente:

- $\bar{A}$  significa que el estatus no es activo (status = "active")
- $\bar{B}$  significa que los puntos no son mayores a 100, es decir son menores o iguales (points  $\leq$  100).

5. Reemplazamos el operador interno and por un or.

Resultado simplificado en código:

```
if status != "active" or points <= 100
```

### 3. TEOREMA DE EXPANSIÓN DE SHANNON (SHANNON'S EXPANSION THEOREM)

#### Definición y Explicación

Propuesto por Claude Shannon, este teorema establece que cualquier función booleana compleja puede ser descompuesta en subfunciones más pequeñas mediante el aislamiento de una variable de control.

#### Identidad Matemática

$$F(X_1, X_2, \dots, X_n) = X_1 \cdot F(1, X_2, \dots, X_n) + X_1' \cdot F(0, X_2, \dots, X_n)$$

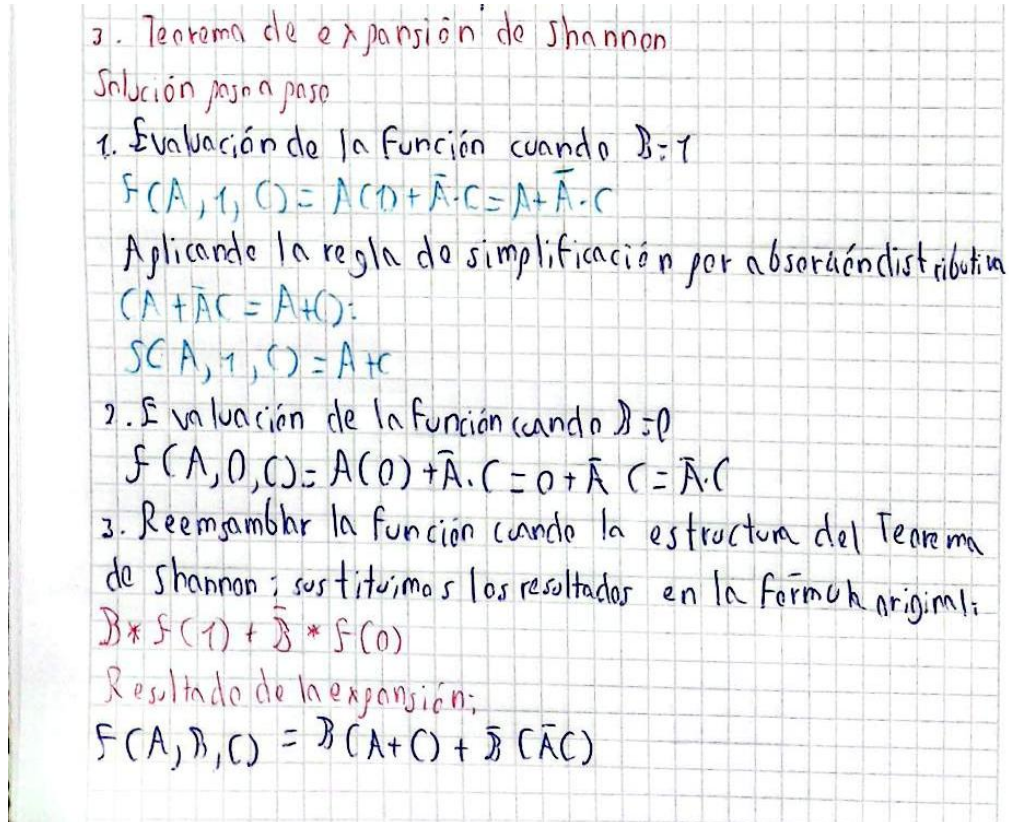
#### Ejercicio Práctico Resuelto

**Enunciado:** Expandir la siguiente función lógica con respecto a la variable B:

$$F(A, B, C) = A \cdot B + A' \cdot C$$



## Solución Paso a Paso



3. Teorema de expansión de Shannon

Solución paso a paso

1. Evaluación de la función cuando  $B=1$

$$F(A, 1, C) = A(1) + \bar{A} \cdot C = A + \bar{A} \cdot C$$

Aplicando la regla de simplificación por absorción distributiva ( $A + \bar{A}C = A + C$ ):

$$S(A, 1, C) = A + C$$

2. Evaluación de la función cuando  $B=0$

$$F(A, 0, C) = A(0) + \bar{A} \cdot C = 0 + \bar{A} \cdot C = \bar{A} \cdot C$$

3. Reemplazar la función cuando la estructura del Teorema de Shannon; sustituimos los resultados en la fórmula original:

$$B * F(1) + \bar{B} * F(0)$$

Resultado de la expansión:

$$F(A, B, C) = B(A + C) + \bar{B}(\bar{A}C)$$

### Resultado de la Expansión

$$F(A, B, C) = B \cdot (A + C) + B' \cdot (A' \cdot C)$$

### Aplicación en la Vida Real y Conexión con Arquitectura de Computadores

En la ecuación resultante de la expansión, la variable B funciona exactamente como la línea de selección (Select) de un multiplexor de 2 a 1:

- Si  $B = 1$ , el multiplexor da paso a los datos de la subfunción  $(A + C)$ .
- Si  $B = 0$ , el multiplexor da paso a los datos de la subfunción  $(A' \cdot C)$ .

Esta propiedad es crítica para las FPGAs (Field Programmable Gate Arrays) y chips programables modernos, donde las funciones lógicas se implementan mediante LUTs (Look-Up Tables) basadas en el Teorema de Shannon.

## 5.2 FASE 5: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales)

### Ejercicio 1 - COMPUERTAS NAND

**Ejercicio #1**

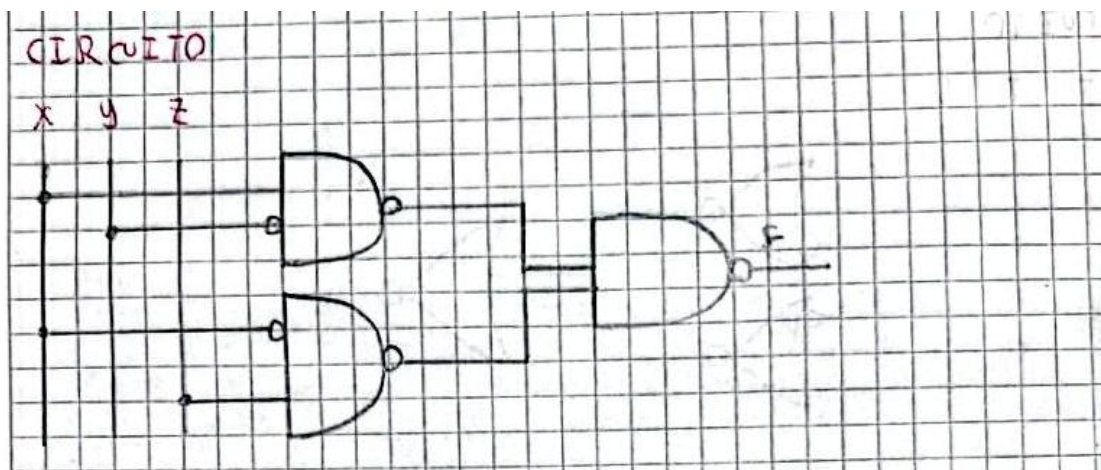
Función original  $\Rightarrow F = x\bar{y} + \bar{x}z$

$F = x\bar{y} + \bar{x}z$

$F = \overline{\overline{x\bar{y} + \bar{x}z}}$  Doble negación

$F = (\overline{x\bar{y}}) \cdot (\overline{\bar{x}z})$  Ley de Morgan

#### Circuito



### Ejercicio 2 – COMPUERTAS NAND

**Ejercicio #2**

Función original  $\Rightarrow F = AB\bar{C} + \bar{A}CD + D\bar{D}$

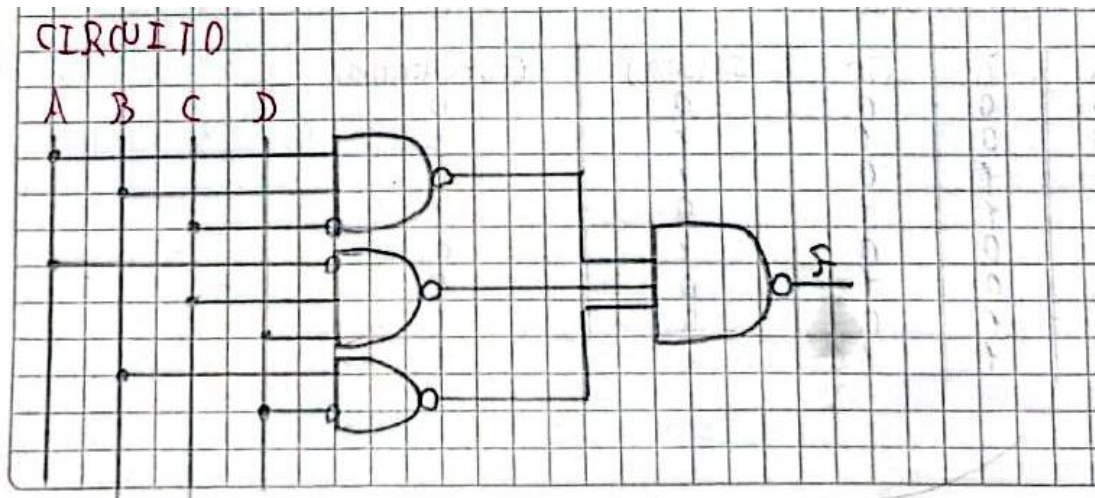
$F = AB\bar{C} + \bar{A}CD + D\bar{D}$

$F = \overline{\overline{AB\bar{C} + \bar{A}CD + D\bar{D}}}$  Doble negación

$F = (\overline{AB\bar{C}}) \cdot (\overline{\bar{A}CD}) \cdot (\overline{D\bar{D}})$  Ley de Morgan



### Circuito



### Ejercicio 3 – COMPUERTAS XOR

Ejercicio #3

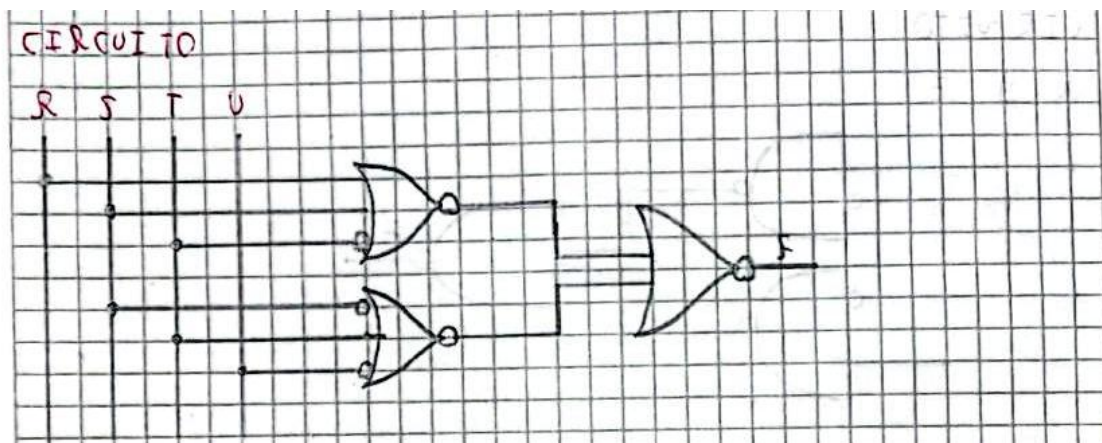
Función original  $\Rightarrow F = (R + S + \bar{T})(\bar{S} + T + \bar{U})$

$F = (R + S + \bar{T})(\bar{S} + T + \bar{U})$

$F = (R + S + \bar{T})(\bar{S} + T + \bar{U})$  Doble negación

$F = (R + S + T) + (\bar{S} + T + \bar{U})$  Ley de Morgan

### Circuito



### **5.3 FASE 5: Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales**

#### **Informe Reflexivo: Optimización Lógica en Sistemas Computacionales y su Impacto Ambiental**

##### **Introducción**

La optimización lógica es una técnica fundamental en el diseño de sistemas computacionales y circuitos digitales. Consiste en simplificar expresiones booleanas para reducir la cantidad de compuertas lógicas y componentes electrónicos necesarios para implementar una determinada función. Este proceso no solo mejora el rendimiento de los sistemas, sino que también contribuye a disminuir costos de fabricación, consumo energético y efectos ambientales asociados a la producción tecnológica.

##### **Desarrollo**

Los sistemas computacionales modernos dependen de millones de circuitos electrónicos que procesan información mediante operaciones lógicas. Cuando una expresión booleana se simplifica utilizando métodos como el Álgebra de Boole o los Mapas de Karnaugh, se reduce el número de compuertas requeridas para construir un circuito. Esta reducción implica un diseño más eficiente y menos complejo.

Desde el punto de vista económico, la optimización lógica permite disminuir la cantidad de materiales utilizados durante la fabricación de circuitos integrados. Al necesitar menos componentes electrónicos, los costos de producción, ensamblaje y mantenimiento también se reducen. Esto resulta especialmente importante en la industria tecnológica, donde pequeñas mejoras en el diseño pueden generar grandes ahorros cuando se fabrican miles o millones de dispositivos.

En cuanto al consumo energético, un circuito simplificado requiere menos transistores activos para realizar la misma función. Como consecuencia, se reduce la energía eléctrica consumida durante el funcionamiento del sistema. Esta característica es fundamental en dispositivos portátiles, centros de datos y equipos de procesamiento de alta demanda, donde la eficiencia energética representa un factor crítico para el rendimiento y la sostenibilidad.

Además, la optimización lógica tiene una relación directa con la reducción de la huella ambiental. La fabricación de componentes electrónicos requiere materias primas, energía y procesos industriales que generan emisiones contaminantes. Al disminuir la cantidad de elementos necesarios para construir un circuito, también se reduce el consumo de recursos naturales y la generación de residuos electrónicos. De esta manera, la optimización lógica contribuye indirectamente a la protección del medio ambiente y al desarrollo de tecnologías más sostenibles.

##### **Reflexión Crítica**

El análisis realizado permite comprender que la optimización lógica no debe considerarse únicamente como una herramienta matemática o técnica para simplificar circuitos. Su impacto trasciende el ámbito computacional y alcanza aspectos económicos, energéticos y ambientales. Como estudiante de Computación, considero que aprender técnicas de simplificación booleana es esencial, ya que permite desarrollar soluciones más eficientes y responsables.

En la actualidad, donde el uso de la tecnología crece constantemente, los profesionales de la informática y la electrónica tienen la responsabilidad de diseñar sistemas que aprovechen mejor los recursos disponibles. Cada compuerta eliminada mediante una



---

correcta optimización representa una oportunidad para reducir costos, ahorrar energía y disminuir el impacto ambiental de los dispositivos tecnológicos.

### **Conclusión**

La optimización lógica constituye una herramienta clave en el diseño de sistemas computacionales modernos. La simplificación de expresiones booleanas permite reducir la complejidad de los circuitos, disminuir costos de fabricación y mejorar la eficiencia energética de los dispositivos. Asimismo, contribuye a reducir la huella ambiental al requerir menos materiales y recursos durante los procesos de producción. Por estas razones, la optimización lógica no solo representa una mejora técnica, sino también una práctica alineada con los principios de sostenibilidad y responsabilidad tecnológica que deben caracterizar a los profesionales de la Computación.

**5.4 Práctica integradora:** cada equipo resuelve un caso aplicado a la computación (diseño de un sumador binario de 1 bit, un comparador digital, o un codificador/decodificador) aplicando todas las técnicas de simplificación aprendidas.

### Diseño de un sumador binario de 1 bit

• Diseño de un sumador binario de 1 bit

1- Introducción:

Un sumador completo de 1 bit es un circuito combinatorial que permite sumar dos bits de entrada y genera un acarreo de entrada (Cin) generado por una suma previa. El circuito genera:

- S = suma
- Cout = Acarreo de salida

### Tabla de Verdad

2- Tabla de verdad

| A | B | Cin | S (suma) | Cout (Acarreo) |
|---|---|-----|----------|----------------|
| 0 | 0 | 0   | 0        | 0              |
| 0 | 0 | 1   | 1        | 0              |
| 0 | 1 | 0   | 1        | 0              |
| 0 | 1 | 1   | 0        | 1              |
| 1 | 0 | 0   | 1        | 0              |
| 1 | 0 | 1   | 0        | 1              |
| 1 | 1 | 0   | 0        | 1              |
| 1 | 1 | 1   | 1        | 1              |



## Función booleana

3- Función booleana

- Función booleana (Función de la suma (S)):  

$$S = \overline{A}B\overline{C}in + A\overline{B}\overline{C}in + A\overline{B}Cin + A\overline{B}Cin$$
- Función del Acarreo (Cout):  

$$Cout = \overline{A}B\overline{C}in + A\overline{B}Cin + A\overline{B}Cin + A\overline{B}Cin$$

## Simplificación con mapas de Karnaugh

4- Simplificación mediante mapas de Karnaugh

- Mapa para S

|   |      |    |    |    |    |
|---|------|----|----|----|----|
|   | BCin | 00 | 01 | 11 | 10 |
| A |      |    |    |    |    |
| 0 |      | 0  | 1  | 0  | 1  |
| 1 |      | 1  | 0  | 1  | 0  |

Análisis:  
 La distribución de los "1" no permite formar agrupaciones que reduzcan el número de variables de la función. Por esta razón, la función de suma se representa de forma más compacta mediante una función de suma se representa de forma más compacta mediante una operación XOR de tres variables.

$$S = A \oplus B \oplus Cin$$

- Mapa de Karnaugh para Cout

|   |      |    |    |    |    |
|---|------|----|----|----|----|
|   | BCin | 00 | 01 | 11 | 10 |
| A |      |    |    |    |    |
| 0 |      | 0  | 0  | 1  | 0  |
| 1 |      | 0  | 1  | 1  | 1  |

Agrupaciones:

- Grupo 1: m5, m7  $\rightarrow B\overline{C}in$
- Grupo 2: m5, m7  $\rightarrow A\overline{C}in$
- Grupo 3: m6, m7  $\rightarrow AB$

Por lo tanto:

$$Cout = AB + A\overline{C}in + B\overline{C}in$$



## Algebra de boole

5.- Simplificación por algebra de Boole

• Función S

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + AB C_{in}$$

$$S = \bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in})$$

Agrupación

Identificación de XOR:

$$\bar{B}C_{in} + B\bar{C}_{in} = B \oplus C_{in}$$

$$\bar{B}\bar{C}_{in} + BC_{in} = \overline{(B \oplus C_{in})}$$

Sustituyendo:

$$S = \bar{A}(B \oplus C_{in}) + A\overline{(B \oplus C_{in})}$$

Definición de XOR

$$S = A \oplus (B \oplus C_{in})$$

Asociativa

$$S = A \oplus B \oplus C_{in}$$

• Función Cout

$$C_{out} = \bar{A}\bar{B}C_{in} + \bar{A}B C_{in} + A\bar{B}C_{in} + AB C_{in}$$

Agrupación

$$C_{out} = C_{in}(\bar{A}\bar{B} + A\bar{B} + A\bar{B}C_{in} + C_{in})$$

Complemento y de función de XOR

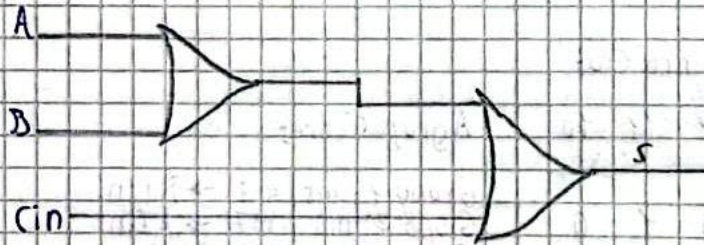
$$C_{out} = C_{in}(\overline{A \oplus B}) + AB$$

$$C_{out} = \bar{A}\bar{B} + AC_{in} + BC_{in}$$

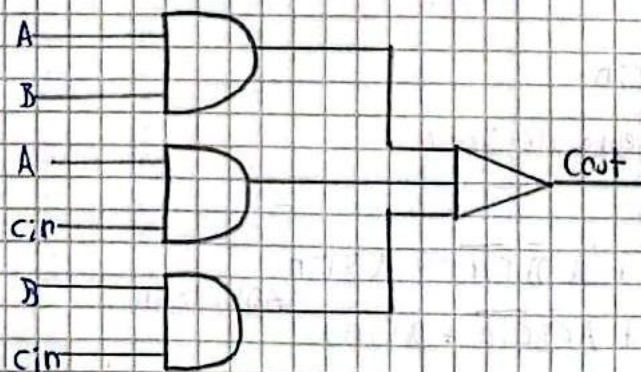
### Diseño del circuito

#### 6. - Diseño del circuito

- Diseño S: 2 compuertas XOR



- Diseño Cout: AND de dos entradas y una compuerta OR de tres entradas

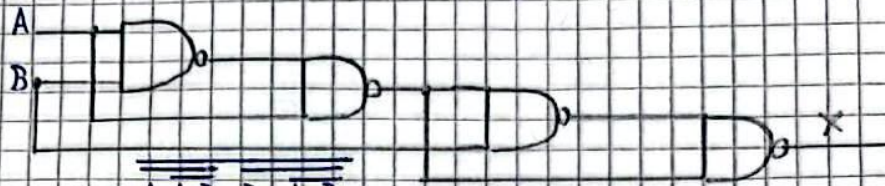




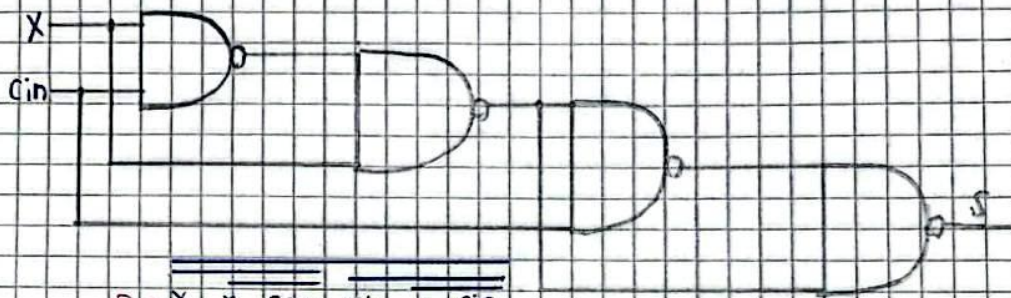
### 7. Implementación con componentes NAND

• Diseño  $S = A \oplus B \oplus C_{in}$

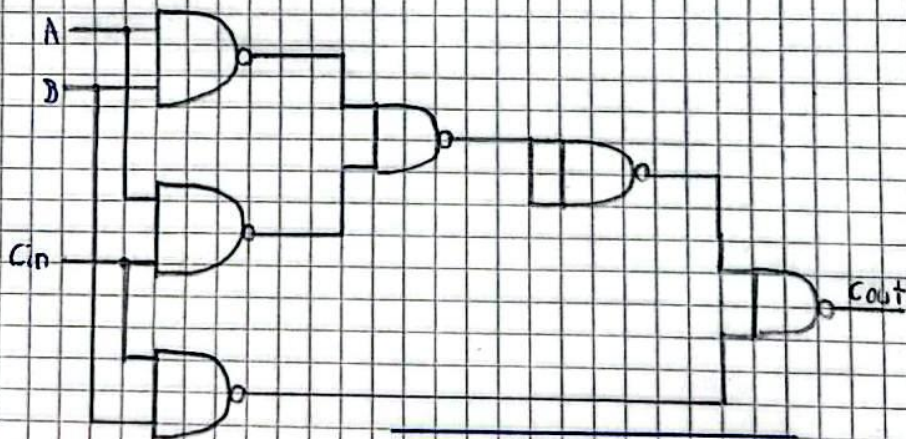
XOR1:  $X = A \oplus B$



$X = A.A.B.B.A.B$   
XOR2:  $S = X \oplus C_{in}$



$S = X.X.C_{in}.C_{in}.X.C_{in}$   
• Salida acarreo  $cout = AB + AC_{in} + BC_{in}$



$Cout = (A.B.A.C_{in}).B.C_{in}$



---

## 5.5 FASE 6: Los equipos entregan el portafolio integrador de la unidad

**Jhoan Rojas:**

<https://drive.google.com/drive/u/3/folders/1YJFcmpSSWU2K24wGGAe3mpxi5JfTdRg3>

**Tyrone Encarnación:**

<https://github.com/tyroneencar8767-ship-it/portafolioDigitalMD/blob/main/README.md#2-evidencias-de-ejercicios>

**Matias Ortega:**

[https://drive.google.com/drive/folders/1AvS\\_OQAKhYvWEwmBj4\\_iMgUa0mfNaVlf?usp=sharing](https://drive.google.com/drive/folders/1AvS_OQAKhYvWEwmBj4_iMgUa0mfNaVlf?usp=sharing)

**Erick Andrade:**

<https://github.com/Blooml2/portafolio-de-matematicas/blob/main/portafolio>

## 6. Preguntas de Control

### 6.1 ¿Qué aprendimos en esta Unidad?

**Erick Andrade**

Tras culminar los APes, logré un dominio más profundo de los fundamentos del diseño digital, específicamente en álgebra de Boole, mapas de Karnaugh. La principal lección fue entender que la simplificación de circuitos no es un proceso rígido: existen múltiples soluciones para un mismo desafío, y es nuestro deber seleccionar la herramienta óptima basados en el contexto. El aprendizaje clave fue no cerrarse a metodologías convencionales; cuando las expresiones lógicas se vuelven complejas y con múltiples términos, el uso de métodos avanzados como los mapas de Karnaugh



resulta crucial, ya que elimina el error humano en la simplificación y agiliza el diseño de hardware de manera mucho más eficiente que la lógica matemática elemental.

### **Tyrone Encarnación**

La culminación de las actividades prácticas me permitió consolidar una comprensión profunda de los pilares del diseño digital, enfocándome con especial atención en el álgebra de Boole y los mapas de Karnaugh. A lo largo de esta unidad, aprendí a implementar propiedades avanzadas como el teorema del consenso y diversas leyes booleanas, las cuales resultan fundamentales para reducir expresiones lógicas y optimizar circuitos sin alterar su comportamiento original.

La enseñanza más valiosa de esta experiencia fue constatar que la simplificación de sistemas no es un proceso rígido ni de un solo camino. Un mismo desafío puede resolverse por distintas vías, lo que nos obliga a evaluar y elegir la estrategia más adecuada según el contexto. En ese sentido, un descubrimiento clave fue la importancia de la flexibilidad metodológica: ante funciones algebraicas densas y con múltiples variables, el uso de herramientas sofisticadas como los mapas de Karnaugh se vuelve indispensable, ya que minimiza el error humano y agiliza la estructuración de hardware de manera mucho más eficiente que la lógica matemática elemental.

Finalmente, el aprendizaje se complementó con el dominio de las compuertas NAND y NOR como bloques universales. Comprender que cualquier función lógica puede construirse utilizando únicamente uno de estos dos tipos de compuertas abrió una nueva perspectiva práctica. A través de los ejercicios, logré transformar circuitos convencionales en diseños optimizados basados exclusivamente en redes NAND o NOR, lo que demuestra cómo la teoría abstracta se traduce directamente en soluciones eficientes y reales para el desarrollo de hardware.

### **Matías Ortega**

Durante el desarrollo de esta unidad aprendí a aplicar diferentes propiedades avanzadas del álgebra de Boole para simplificar funciones lógicas y optimizar circuitos digitales. Comprendí el funcionamiento del teorema del consenso y otras leyes booleanas que permiten reducir expresiones sin alterar su comportamiento lógico. Además, aprendí que las compuertas NAND y NOR son consideradas

compuertas universales, ya que es posible construir cualquier función lógica utilizando únicamente uno de estos tipos de compuertas. A través de ejercicios prácticos pude transformar circuitos convencionales en circuitos implementados solo con NAND o solo con NOR. También conocí conceptos como la expansión de Shannon y la resolución booleana, herramientas utilizadas en el análisis y diseño de sistemas digitales más complejos. Asimismo, reforcé mis conocimientos sobre mapas de Karnaugh y simplificación algebraica para obtener diseños más eficientes.

Finalmente, comprendí la importancia de la optimización lógica en los sistemas computacionales, ya que la reducción de compuertas y componentes electrónicos permite disminuir costos de fabricación, consumo energético y el impacto ambiental asociado a la producción de circuitos electrónicos. Estos conocimientos fortalecen mi formación en Computación al relacionar la teoría matemática con aplicaciones reales en hardware y software.

## 7. Conclusiones

Se concluye que:

- Se demostró que el uso de herramientas avanzadas como los mapas de Karnaugh y el teorema del consenso agiliza el diseño lógico y elimina el error humano, probando que la simplificación eficiente es clave para transformar la abstracción matemática en arquitecturas de hardware optimizadas.
- La implementación de circuitos mediante compuertas universales (NAND/NOR) evidenció que la reducción de expresiones booleanas tiene un impacto físico real, ya que al disminuir el número de transistores se reducen los costos de manufactura, el consumo energético y la huella ambiental.



## 8. Reflexiones críticas personales

### Andrade Erick

El desarrollo de estas prácticas me permitió entender que la simplificación de circuitos no es un mero ejercicio matemático abstracto, sino el pilar fundamental de la optimización de hardware. En el diseño digital moderno, una expresión booleana sin simplificar se traduce directamente en un circuito sobredimensionado, con un exceso de compuertas lógicas y conexiones innecesarias.

Comprendí que cada paso de reducción algebraica o mediante mapas de Karnaugh tiene un impacto real en el mundo físico: al eliminar términos redundantes (como en el Teorema del Consenso), se reduce el número de transistores en el silicio, lo que disminuye el área del chip, reduce los costos de manufactura y, de manera crítica, minimiza el consumo energético y la disipación de calor del sistema.

### Tyrone Encarnación

El desarrollo de esta unidad me permitió comprender que el diseño digital no radica en la aplicación aislada de fórmulas, sino en la búsqueda estratégica de la eficiencia física a través de la abstracción matemática, donde la simplificación de circuitos mediante el álgebra de Boole y los mapas de Karnaugh deja de ser un mero ejercicio conceptual para convertirse en el pilar fundamental del desarrollo de hardware moderno. A través de las prácticas, asimilé que la optimización carece de un camino rígido y que, ante expresiones complejas, la flexibilidad metodológica es crucial; herramientas avanzadas como el teorema del consenso o los diagramas de Karnaugh no solo mitigan el error humano, sino que agilizan el diseño con una eficiencia muy superior a la lógica elemental. Este proceso de reducción impacta directamente en el mundo real, ya que cada término o redundancia eliminada se traduce en una disminución de compuertas lógicas y conexiones innecesarias en el silicio, lo que reduce el número de transistores, optimiza el área del chip, abarata los costos de manufactura y, de manera crítica, minimiza el consumo energético y la disipación térmica. Asimismo, esta visión se potencia al dominar la universalidad de las compuertas NAND y NOR, cuya capacidad para replicar cualquier función lógica demuestra cómo la reestructuración y estandarización de circuitos convencionales hacia un solo tipo de componente optimiza la producción industrial, demostrando en última

instancia que el verdadero diseño digital exige un balance perfecto entre la destreza matemática para sintetizar y la conciencia técnica de sus repercusiones físicas.

### **Matías Ortega**

El estudio de las propiedades adicionales del álgebra de Boole me permitió comprender que la optimización lógica es un aspecto fundamental en el desarrollo de sistemas computacionales. A lo largo de las actividades realizadas, observé que una misma función puede implementarse de diferentes maneras, pero mediante técnicas de simplificación es posible obtener soluciones más eficientes y con menor cantidad de componentes.

Uno de los aspectos que más llamó mi atención fue el uso de compuertas NAND y NOR como puertas universales. Aunque al principio parecía un proceso complejo debido a las negaciones y transformaciones necesarias, posteriormente entendí que estas técnicas permiten estandarizar el diseño de circuitos y facilitar su implementación práctica. Además, comprobé que la simplificación lógica no solo tiene beneficios académicos, sino también aplicaciones reales en la industria tecnológica, donde reducir componentes significa disminuir costos de producción, consumo de energía y generación de residuos electrónicos.

Considero que este tema es especialmente importante para la formación de un profesional en Computación, ya que desarrolla habilidades de análisis, razonamiento lógico y resolución de problemas. Más allá de obtener una respuesta correcta, aprendí a buscar soluciones más eficientes y optimizadas, una competencia que será útil tanto en el diseño de hardware como en el desarrollo de software. En conclusión, esta experiencia fortaleció mi comprensión de la lógica digital y me permitió valorar la importancia de la optimización como una herramienta clave para crear tecnologías más eficientes y sostenibles.

### **Jhoan Rojas**

El estudio del álgebra de Boole y la simplificación lógica no debe percibirse como un proceso abstracto, ya que su verdadero valor radica en su aplicación directa en los circuitos lógicos, el diseño digital, las consultas SQL y la programación. Al abordar el diseño práctico de un sumador binario de 1 bit o la configuración de circuitos usando únicamente compuertas universales NAND o NOR, se evidencia que la minimización de





funciones es una necesidad operativa real dentro de la computación. Una reducción eficiente de las expresiones booleanas permite disminuir de forma drástica la cantidad de compuertas en el hardware, lo cual abarata directamente los costos de fabricación industrial. Además, este proceso de optimización lógica disminuye de forma significativa el consumo energético de los sistemas computacionales y reduce la huella ambiental asociada a la manufactura de circuitos. De este modo, la abstracción matemática se consolida como una herramienta indispensable para desarrollar tecnologías eficientes, económicamente viables y ambientalmente responsables bajo un ejercicio profesional honesto y consciente.

## 9. Bibliografía / Referencias

- [1]Oscar Levin, University of Northern Colorado (2022). Discrete Mathematics: An Open Introduction – 4th Edition. Universidad Tecnológica de Pereira. ISBN: 9781534970748.
- [2]Harris Kwon (2024). A Spiral Workbook for Discrete Mathematics. Milne Open Textbooks. ISBN: 978-1-956862-01-0.
- [3]André Greiner-Petter (2023). Making Presentation Math Computable. Springer. ISBN: 978-3-658-40472-7.
- [4]Kevin Powell (2025). Discrete Perspectives in Mathematics – Second Edition.
- [5]Kazushi Ikeda et al. (2024). Advanced Mathematical Science for Mobility Society. Springer. ISBN: 978-981-99-9771-8.

---

## 10. Anexos

### Matías Ortega

#### 5.1 FASE 5: Los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos.

#### GUÍA DE INVESTIGACIÓN Y PRESENTACIÓN: LEYES Y PROPIEDADES ADICIONALES DEL ÁLGEBRA BOOLEANA

##### 1. TEOREMA DEL CONSENSO DUAL (DUAL CONSENSUS THEOREM)

###### Definición y Explicación

Mientras que el teorema del consenso estándar opera sobre una suma de productos, el Teorema del Consenso Dual se aplica directamente a expresiones en forma de producto de sumas (POS). Este teorema establece que en una combinación de tres variables lógicas sumadas en parejas, uno de los términos de la multiplicación es completamente redundante y puede ser eliminado sin alterar el comportamiento o resultado de la función.

###### Identidad Matemática

$$(A + B) \cdot (A' + C) \cdot (B + C) = (A + B) \cdot (A' + C)$$

Donde A' significa "A negado".

###### Demostración intuitiva

Analicemos el término  $(B + C)$ . Si tanto B como C son verdaderos (1), este paréntesis es verdadero. Sin embargo, debido a que la variable A debe tomar obligatoriamente un valor binario (ya sea 1 o 0), su presencia en los dos primeros términos  $((A + B) \text{ y } (A' + C))$  forzará a que uno de ellos dependa exclusivamente de los valores de B o C. Esto hace que el tercer paréntesis sea una repetición lógica que no aporta nueva información.

###### Ejercicio Práctico Resuelto

**Enunciado:** Simplificar la siguiente expresión lógica utilizada para el control de un circuito de automatización:

$$F = (X + Y) \cdot (X' + Z) \cdot (Y + Z)$$

## Solución Paso a Paso

### Reglas y propiedades adicionales del álgebra booleana

Ejercicio Práctico Resuelto:

Simplificar la siguiente expresión lógica utilizada para el control de un circuito de automatización:

$$F = (X + Y) (\bar{X} + Z) (Y + Z)$$

Solución Paso a Paso:

1. Identificamos las variables correspondientes a la identidad del teorema

$$A = X, B = Y, C = Z$$

2. Buscamos el término de consenso redundante, el cual está formado por las variables que acompañan a la variable que cambia de estado ( $X$  y  $\bar{X}$ ). En este caso, las variables acompañantes son  $Y$  y  $Z$ .

3. El término de la expresión

4. Eliminamos el término de la expresión.

Resultado Simplificado:  $F = (X + Y) (\bar{X} + Z)$

### Resultado Simplificado:

$$F = (X + Y) \cdot (X' + Z)$$

### Aplicación en la Vida Real y Conexión con Diseño Digital

En el diseño de hardware y circuitos lógicos integrados, implementar la función original:

$$F = (X + Y) \cdot (X' + Z) \cdot (Y + Z)$$

requiere físicamente tres compuertas OR independientes que luego conectan sus salidas a una compuerta AND de tres entradas.

Al aplicar el Teorema del Consenso Dual, optimizamos el circuito reduciéndolo a sólo dos compuertas OR y una compuerta AND de dos entradas.

## 2. ÁLGEBRA BOOLEANA GENERAL: LEYES DE DE MORGAN

### Definición y Explicación

Las Leyes de De Morgan son herramientas fundamentales para la transformación de compuertas lógicas y la simplificación de condiciones complejas. Explican la equivalencia matemática que existe al distribuir o retirar una negación (operador NOT) de un paréntesis que contiene operaciones OR o AND.

### Identidades Matemáticas

#### Primera Ley:

$$(A \cdot B)' = A' + B'$$

(La multiplicación negada se convierte en suma de variables negadas)

#### Segunda Ley:

$$(A + B)' = A' \cdot B'$$

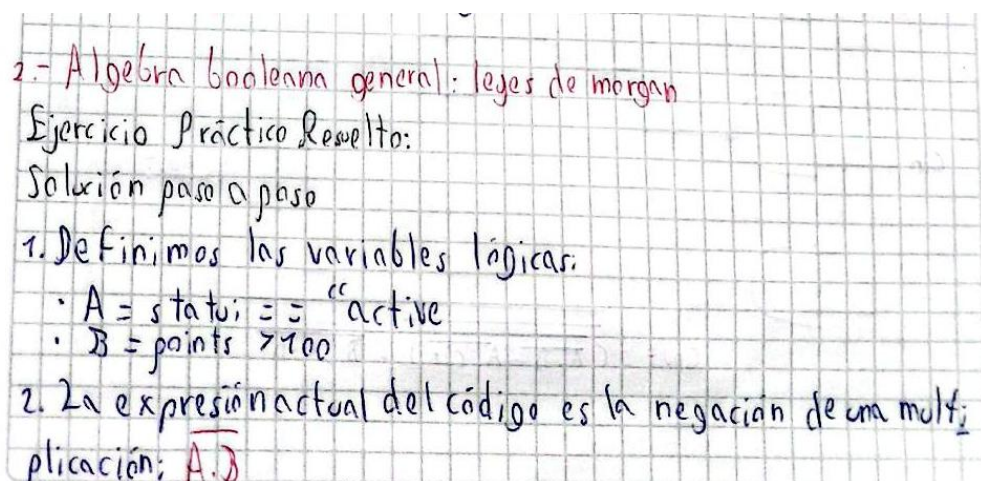
(La suma negada se convierte en multiplicación de variables negadas)

### Ejercicio Práctico Resuelto

**Enunciado:** Un sistema de desarrollo de software cuenta con la siguiente condición lógica anidada:

if not (status == "active" and points > 100):

### Solución Paso a Paso



2.- Álgebra booleana general: leyes de morgan

Ejercicio Práctico Resuelto:

Solución paso a paso

1. Definimos las variables lógicas:

- $A = \text{status} == \text{"active"}$
- $B = \text{points} > 100$

2. La expresión actual del código es la negación de una multiplicación:  $\overline{A \cdot B}$

3. Aplicamos la primera Ley de Morgan:  $\overline{A \cdot B} = \overline{A} + \overline{B}$ .

4. Negamos cada variable individualmente:

- $\overline{A}$  significa que el estatus no es activo (status = "active")
- $\overline{B}$  significa que los puntos no son mayores a 100, es decir son menores o iguales (points  $\leq$  100).

5. Reemplazamos el operador interno and por un or.

Resultado simplificado en código:

```
if status != "active" or points <= 100
```

### 3. TEOREMA DE EXPANSIÓN DE SHANNON (SHANNON'S EXPANSION THEOREM)

#### Definición y Explicación

Propuesto por Claude Shannon, este teorema establece que cualquier función booleana compleja puede ser descompuesta en subfunciones más pequeñas mediante el aislamiento de una variable de control.

#### Identidad Matemática

$$F(X_1, X_2, \dots, X_n) = X_1 \cdot F(1, X_2, \dots, X_n) + X_1' \cdot F(0, X_2, \dots, X_n)$$

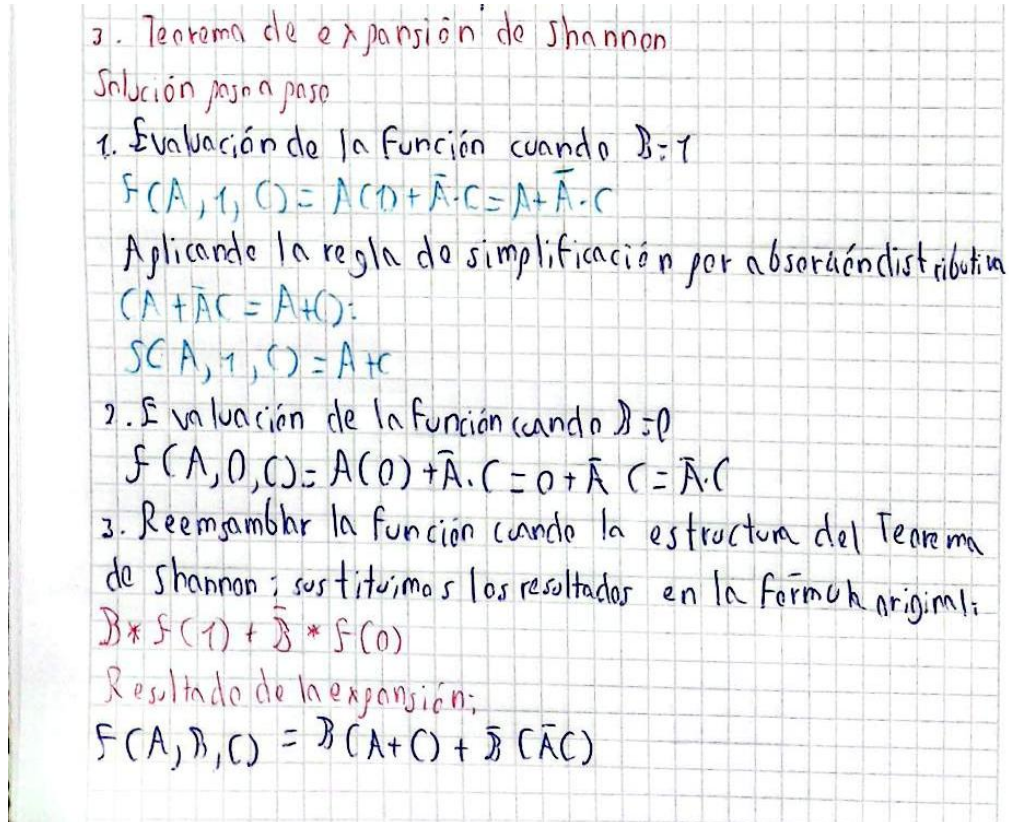
#### Ejercicio Práctico Resuelto

**Enunciado:** Expandir la siguiente función lógica con respecto a la variable B:

$$F(A, B, C) = A \cdot B + A' \cdot C$$



## Solución Paso a Paso



3. Teorema de expansión de Shannon

Solución paso a paso

1. Evaluación de la función cuando  $B=1$

$$F(A, 1, C) = A(1) + \bar{A} \cdot C = A + \bar{A} \cdot C$$

Aplicando la regla de simplificación por absorción distributiva ( $A + \bar{A}C = A + C$ ):

$$S(A, 1, C) = A + C$$

2. Evaluación de la función cuando  $B=0$

$$F(A, 0, C) = A(0) + \bar{A} \cdot C = 0 + \bar{A} \cdot C = \bar{A} \cdot C$$

3. Reemplazar la función cuando la estructura del Teorema de Shannon; sustituimos los resultados en la fórmula original:

$$B * F(1) + \bar{B} * F(0)$$

Resultado de la expansión:

$$F(A, B, C) = B(A + C) + \bar{B}(\bar{A}C)$$

### Resultado de la Expansión

$$F(A, B, C) = B \cdot (A + C) + B' \cdot (A' \cdot C)$$

### Aplicación en la Vida Real y Conexión con Arquitectura de Computadores

En la ecuación resultante de la expansión, la variable B funciona exactamente como la línea de selección (Select) de un multiplexor de 2 a 1:

- Si  $B = 1$ , el multiplexor da paso a los datos de la subfunción  $(A + C)$ .
- Si  $B = 0$ , el multiplexor da paso a los datos de la subfunción  $(A' \cdot C)$ .

Esta propiedad es crítica para las FPGAs (Field Programmable Gate Arrays) y chips programables modernos, donde las funciones lógicas se implementan mediante LUTs (Look-Up Tables) basadas en el Teorema de Shannon.

## 5.2 FASE 5: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales)

### Ejercicio 1 - COMPUERTAS NAND

**Ejercicio #1**

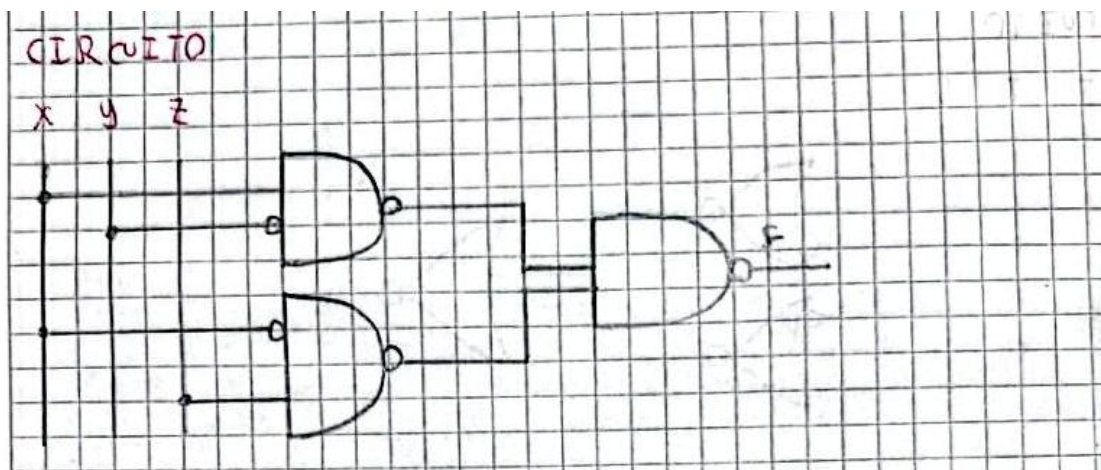
Función original  $\Rightarrow F = x\bar{y} + \bar{x}z$

$F = x\bar{y} + \bar{x}z$

$F = \overline{\overline{x\bar{y} + \bar{x}z}}$  Doble negación

$F = (\overline{x\bar{y}}) \cdot (\overline{\bar{x}z})$  Ley de Morgan

#### Circuito



### Ejercicio 2 – COMPUERTAS NAND

**Ejercicio #2**

Función original  $\Rightarrow F = AB\bar{C} + \bar{A}CD + D\bar{D}$

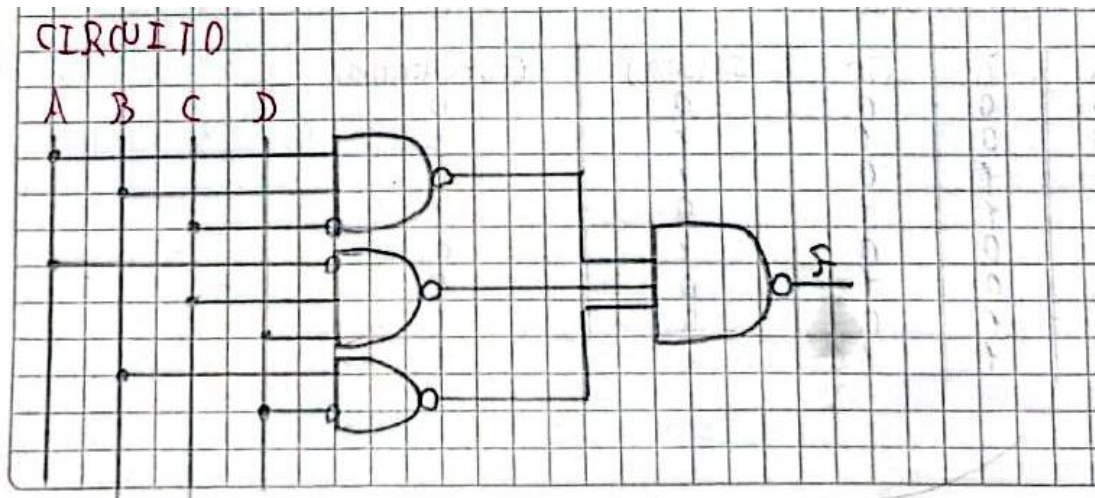
$F = AB\bar{C} + \bar{A}CD + D\bar{D}$

$F = \overline{\overline{AB\bar{C} + \bar{A}CD + D\bar{D}}}$  Doble negación

$F = (\overline{AB\bar{C}}) \cdot (\overline{\bar{A}CD}) \cdot (\overline{D\bar{D}})$  Ley de Morgan



### Circuito



### Ejercicio 3 – COMPUERTAS XOR

Ejercicio #3

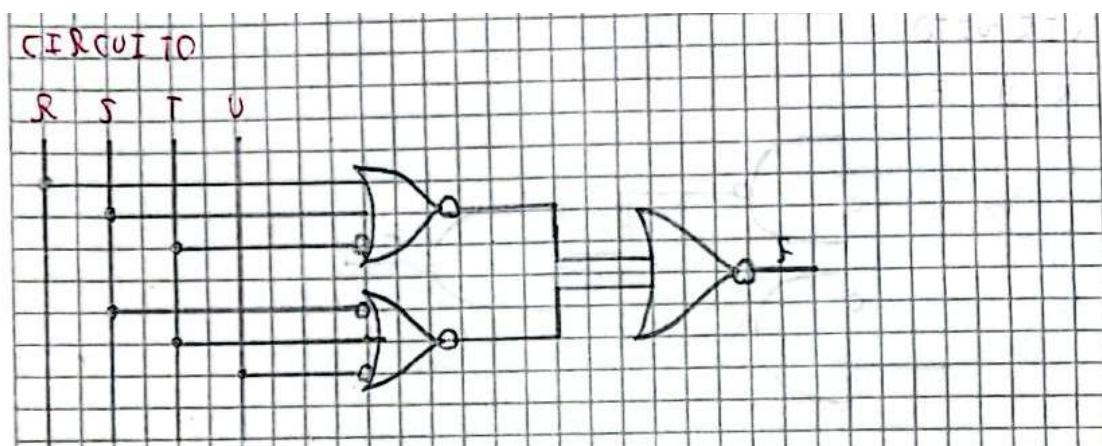
Función original  $\Rightarrow F = (R + S + \bar{T})(\bar{S} + T + \bar{U})$

$F = (R + S + \bar{T})(\bar{S} + T + \bar{U})$

$F = (R + S + \bar{T})(\bar{S} + T + \bar{U})$  Doble negación

$F = (R + S + T) + (\bar{S} + T + \bar{U})$  Ley de Morgan

### Circuito



### **5.3 FASE 5: Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales**

#### **Informe Reflexivo: Optimización Lógica en Sistemas Computacionales y su Impacto Ambiental**

##### **Introducción**

La optimización lógica es una técnica fundamental en el diseño de sistemas computacionales y circuitos digitales. Consiste en simplificar expresiones booleanas para reducir la cantidad de compuertas lógicas y componentes electrónicos necesarios para implementar una determinada función. Este proceso no solo mejora el rendimiento de los sistemas, sino que también contribuye a disminuir costos de fabricación, consumo energético y efectos ambientales asociados a la producción tecnológica.

##### **Desarrollo**

Los sistemas computacionales modernos dependen de millones de circuitos electrónicos que procesan información mediante operaciones lógicas. Cuando una expresión booleana se simplifica utilizando métodos como el Álgebra de Boole o los Mapas de Karnaugh, se reduce el número de compuertas requeridas para construir un circuito. Esta reducción implica un diseño más eficiente y menos complejo.

Desde el punto de vista económico, la optimización lógica permite disminuir la cantidad de materiales utilizados durante la fabricación de circuitos integrados. Al necesitar menos componentes electrónicos, los costos de producción, ensamblaje y mantenimiento también se reducen. Esto resulta especialmente importante en la industria tecnológica, donde pequeñas mejoras en el diseño pueden generar grandes ahorros cuando se fabrican miles o millones de dispositivos.

En cuanto al consumo energético, un circuito simplificado requiere menos transistores activos para realizar la misma función. Como consecuencia, se reduce la energía eléctrica consumida durante el funcionamiento del sistema. Esta característica es fundamental en dispositivos portátiles, centros de datos y equipos de procesamiento de alta demanda, donde la eficiencia energética representa un factor crítico para el rendimiento y la sostenibilidad.

Además, la optimización lógica tiene una relación directa con la reducción de la huella ambiental. La fabricación de componentes electrónicos requiere materias primas, energía y procesos industriales que generan emisiones contaminantes. Al disminuir la cantidad de elementos necesarios para construir un circuito, también se reduce el consumo de recursos naturales y la generación de residuos electrónicos. De esta manera, la optimización lógica contribuye indirectamente a la protección del medio ambiente y al desarrollo de tecnologías más sostenibles.

##### **Reflexión Crítica**

El análisis realizado permite comprender que la optimización lógica no debe considerarse únicamente como una herramienta matemática o técnica para simplificar circuitos. Su impacto trasciende el ámbito computacional y alcanza aspectos económicos, energéticos y ambientales. Como estudiante de Computación, considero que aprender técnicas de simplificación booleana es esencial, ya que permite desarrollar soluciones más eficientes y responsables.

En la actualidad, donde el uso de la tecnología crece constantemente, los profesionales de la informática y la electrónica tienen la responsabilidad de diseñar sistemas que aprovechen mejor los recursos disponibles. Cada compuerta eliminada mediante una



---

correcta optimización representa una oportunidad para reducir costos, ahorrar energía y disminuir el impacto ambiental de los dispositivos tecnológicos.

### **Conclusión**

La optimización lógica constituye una herramienta clave en el diseño de sistemas computacionales modernos. La simplificación de expresiones booleanas permite reducir la complejidad de los circuitos, disminuir costos de fabricación y mejorar la eficiencia energética de los dispositivos. Asimismo, contribuye a reducir la huella ambiental al requerir menos materiales y recursos durante los procesos de producción. Por estas razones, la optimización lógica no solo representa una mejora técnica, sino también una práctica alineada con los principios de sostenibilidad y responsabilidad tecnológica que deben caracterizar a los profesionales de la Computación.



**5.4 Práctica integradora:** cada equipo resuelve un caso aplicado a la computación (diseño de un sumador binario de 1 bit, un comparador digital, o un codificador/decodificador) aplicando todas las técnicas de simplificación aprendidas.

### Diseño de un sumador binario de 1 bit

• Diseño de un sumador binario de 1 bit

1- Introducción:

Un sumador completo de 1 bit es un circuito combinatorial que permite sumar dos bits de entrada y genera un acarreo de entrada (Cin) generado por una suma previa. El circuito genera:

- S = suma
- Cout = Acarreo de salida

### Tabla de Verdad

2- Tabla de verdad

| A | B | Cin | S (suma) | Cout (Acarreo) |
|---|---|-----|----------|----------------|
| 0 | 0 | 0   | 0        | 0              |
| 0 | 0 | 1   | 1        | 0              |
| 0 | 1 | 0   | 1        | 0              |
| 0 | 1 | 1   | 0        | 1              |
| 1 | 0 | 0   | 1        | 0              |
| 1 | 0 | 1   | 0        | 1              |
| 1 | 1 | 0   | 0        | 1              |
| 1 | 1 | 1   | 1        | 1              |

## Función booleana

3- Función booleana

- Función booleana (Función de la suma (S)):  

$$S = \overline{A}B\overline{C}in + \overline{A}B\overline{C}in + A\overline{B}Cin + A\overline{B}Cin$$
- Función del Acarreo (Cout):  

$$Cout = \overline{A}B\overline{C}in + \overline{A}B\overline{C}in + A\overline{B}Cin + A\overline{B}Cin$$

## Simplificación con mapas de Karnaugh

4- Simplificación mediante mapas de Karnaugh

- Mapa para S

|   |      |    |    |    |    |
|---|------|----|----|----|----|
|   | BCin | 00 | 01 | 11 | 10 |
| A |      |    |    |    |    |
| 0 |      | 0  | 1  | 0  | 1  |
| 1 |      | 1  | 0  | 1  | 0  |

Análisis:  
 La distribución de los "1" no permite formar agrupaciones que reduzcan el número de variables de la función. Por esta razón, la función de suma se representa de forma más compacta mediante una función de suma se representa de forma más compacta mediante una operación XOR de tres variables.

$$S = A \oplus B \oplus Cin$$

- Mapa de Karnaugh para Cout

|   |      |    |    |    |    |
|---|------|----|----|----|----|
|   | BCin | 00 | 01 | 11 | 10 |
| A |      |    |    |    |    |
| 0 |      | 0  | 0  | 1  | 0  |
| 1 |      | 0  | 1  | 1  | 1  |

Agrupaciones:

- Grupo 1: m5, m7  $\rightarrow B\overline{C}in$
- Grupo 2: m5, m7  $\rightarrow A\overline{C}in$
- Grupo 3: m6, m7  $\rightarrow AB$

Por lo tanto:

$$Cout = AB + A\overline{C}in + B\overline{C}in$$



## Algebra de boole

5.- Simplificación por algebra de Boole

• Función S

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + AB C_{in}$$

$$S = \bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in})$$

Agrupación

Identificación de XOR:

$$\bar{B}C_{in} + B\bar{C}_{in} = B \oplus C_{in}$$

$$\bar{B}\bar{C}_{in} + BC_{in} = \overline{(B \oplus C_{in})}$$

Sustituyendo:

$$S = \bar{A}(B \oplus C_{in}) + A\overline{(B \oplus C_{in})}$$

Definición de XOR

$$S = A \oplus (B \oplus C_{in})$$

Asociativa

$$S = A \oplus B \oplus C_{in}$$

• Función Cout

$$C_{out} = \bar{A}\bar{B}C_{in} + A\bar{B}C_{in} + A\bar{B}\bar{C}_{in} + AB C_{in}$$

Agrupación

$$C_{out} = C_{in}(\bar{A}\bar{B} + A\bar{B}) + A\bar{B}\bar{C}_{in} + AB C_{in}$$

Complemento y de función de XOR

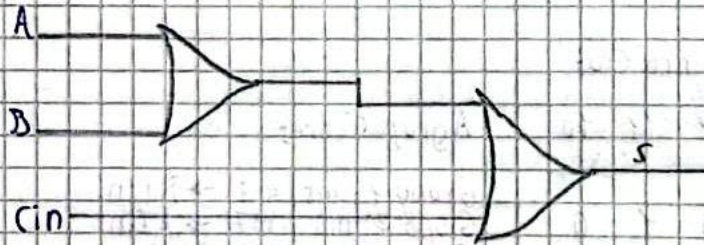
$$C_{out} = C_{in}(\overline{A \oplus B}) + A\bar{B}$$

$$C_{out} = A\bar{B} + AC_{in} + BC_{in}$$

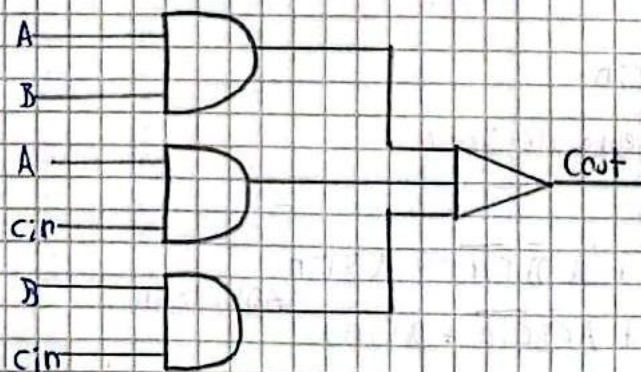
### Diseño del circuito

#### 6. - Diseño del circuito

- Diseño S: 2 compuertas XOR



- Diseño Cout: AND de dos entradas y una compuerta OR de tres entradas

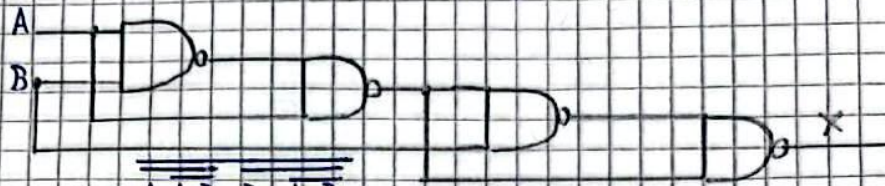




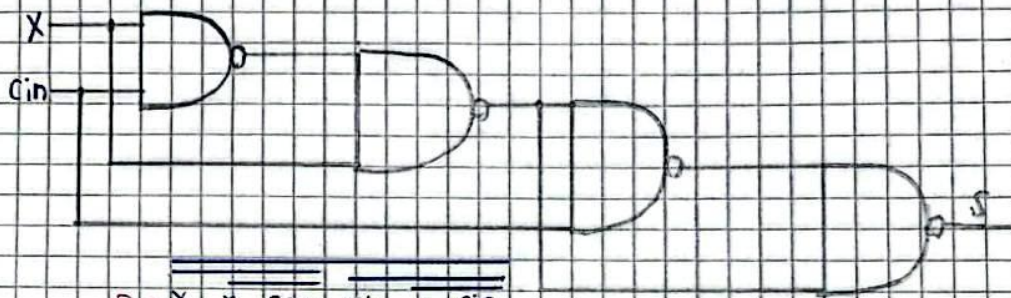
### 7. Implementación con componentes NAND

• Diseño  $S = A \oplus B \oplus C_{in}$

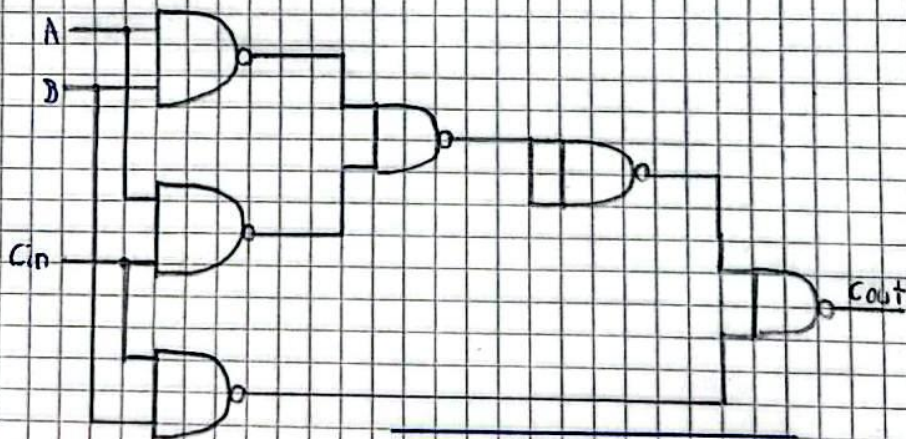
XOR1:  $X = A \oplus B$



$X = A.A.B.B.A.B$   
XOR2:  $S = X \oplus C_{in}$



$S = X.X.C_{in}.C_{in}.X.C_{in}$   
• Salida acarreo  $cout = AB + AC_{in} + BC_{in}$



$Cout = (A.B.A.C_{in}).B.C_{in}$

---

## Erick Andrade

### 5.1 FASE 5: Los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos.

#### GUÍA DE INVESTIGACIÓN Y PRESENTACIÓN: LEYES Y PROPIEDADES ADICIONALES DEL ÁLGEBRA BOOLEANA

##### 1. TEOREMA DEL CONSENSO DUAL (DUAL CONSENSUS THEOREM)

###### Definición y Explicación

Mientras que el teorema del consenso estándar opera sobre una suma de productos, el Teorema del Consenso Dual se aplica directamente a expresiones en forma de producto de sumas (POS). Este teorema establece que en una combinación de tres variables lógicas sumadas en parejas, uno de los términos de la multiplicación es completamente redundante y puede ser eliminado sin alterar el comportamiento o resultado de la función.

###### Identidad Matemática

$$(A + B) \cdot (A' + C) \cdot (B + C) = (A + B) \cdot (A' + C)$$

Donde  $A'$  significa "A negado".

###### Demostración intuitiva

Analicemos el término  $(B + C)$ . Si tanto B como C son verdaderos (1), este paréntesis es verdadero. Sin embargo, debido a que la variable A debe tomar obligatoriamente un valor binario (ya sea 1 o 0), su presencia en los dos primeros términos  $((A + B) \text{ y } (A' + C))$  forzará a que uno de ellos dependa exclusivamente de los valores de B o C. Esto hace que el tercer paréntesis sea una repetición lógica que no aporta nueva información.

###### Ejercicio Práctico Resuelto

**Enunciado:** Simplificar la siguiente expresión lógica utilizada para el control de un circuito de automatización:

$$F = (X + Y) \cdot (X' + Z) \cdot (Y + Z)$$

### Solución Paso a Paso

Ejercicio Práctico Resuelto

Simplificar la siguiente expresión lógica utilizada para el control de un circuito de automatización:

$$F = (X + Y)(\bar{X} + Z)(Y + Z)$$

Solución Paso a Paso:

1 Identificamos las variables correspondientes a la identidad del teorema:

$$A = X, B = Y, C = Z.$$

2 Buscamos el término de consenso redundante, el cual está formado por las variables que acompañan a la variable que cambia de estado ( $X$  y  $\bar{X}$ ). En este caso, las variables acompañantes son  $Y$  y  $Z$ .

3 El término de la expresión,

4 Eliminamos el término de la expresión.

$$\text{Resultado Simplificado: } F = (X + Y)(\bar{X} + Z)$$

### Resultado Simplificado:

$$F = (X + Y) \cdot (X' + Z)$$

### Aplicación en la Vida Real y Conexión con Diseño Digital

En el diseño de hardware y circuitos lógicos integrados, implementar la función original:

$$F = (X + Y) \cdot (X' + Z) \cdot (Y + Z)$$

requiere físicamente tres compuertas OR independientes que luego conectan sus salidas a una compuerta AND de tres entradas.

Al aplicar el Teorema del Consenso Dual, optimizamos el circuito reduciéndolo a sólo dos compuertas OR y una compuerta AND de dos entradas.

## 2. ÁLGEBRA BOOLEANA GENERAL: LEYES DE DE MORGAN

### Definición y Explicación

Las Leyes de De Morgan son herramientas fundamentales para la transformación de compuertas lógicas y la simplificación de condiciones complejas. Explican la equivalencia matemática que existe al distribuir o retirar una negación (operador NOT) de un paréntesis que contiene operaciones OR o AND.

### Identidades Matemáticas

#### Primera Ley:

$$(A \cdot B)' = A' + B'$$

(La multiplicación negada se convierte en suma de variables negadas)

#### Segunda Ley:

$$(A + B)' = A' \cdot B'$$

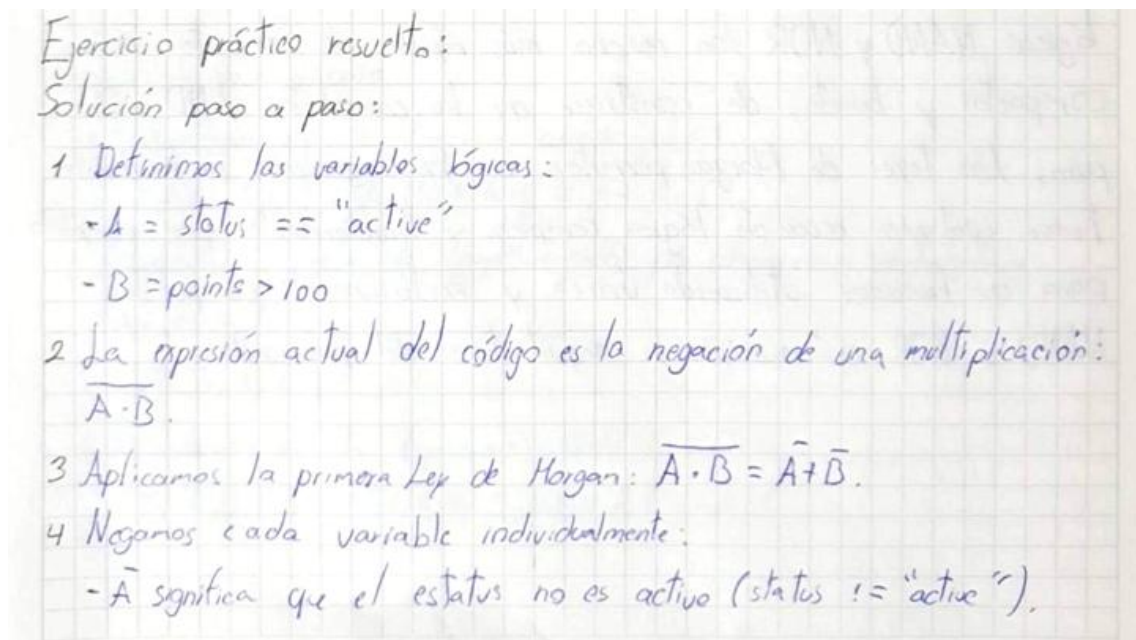
(La suma negada se convierte en multiplicación de variables negadas)

### Ejercicio Práctico Resuelto

**Enunciado:** Un sistema de desarrollo de software cuenta con la siguiente condición lógica anidada:

if not (status == "active" and points > 100):

### Solución Paso a Paso



Ejercicio práctico resuelto:

Solución paso a paso:

- Definimos las variables lógicas:
  - $A = \text{status} == \text{"active"}$
  - $B = \text{points} > 100$
- La expresión actual del código es la negación de una multiplicación:  
 $\overline{A \cdot B}$ .
- Aplicamos la primera Ley de Morgan:  $\overline{A \cdot B} = \overline{A} + \overline{B}$ .
- Negamos cada variable individualmente:
  - $\overline{A}$  significa que el estatus no es activo (status != "active").



-  $\bar{B}$  significa que los puntos no son mayores a 100, es decir, son menores o iguales ( $\text{points} \leq 100$ ).

5 Reemplazamos el operador interno and por un or.

Resultado simplificado en código:

```
if status != "active" or points <= 100:
```

### 3. TEOREMA DE EXPANSIÓN DE SHANNON (SHANNON'S EXPANSION THEOREM)

#### Definición y Explicación

Propuesto por Claude Shannon, este teorema establece que cualquier función booleana compleja puede ser descompuesta en subfunciones más pequeñas mediante el aislamiento de una variable de control.

#### Identidad Matemática

$$F(X_1, X_2, \dots, X_n) = X_1 \cdot F(1, X_2, \dots, X_n) + X_1' \cdot F(0, X_2, \dots, X_n)$$

#### Ejercicio Práctico Resuelto

**Enunciado:** Expandir la siguiente función lógica con respecto a la variable B:

$$F(A, B, C) = A \cdot B + A' \cdot C$$

#### Solución Paso a Paso

Ejercicio práctico resuelto

Expandir la siguiente función lógica con respecto a la variable B:

$$F(A, B, C) = AB + \bar{A}C$$

Solución paso a paso:

1 Evaluación de la función cuando  $B=1$ :

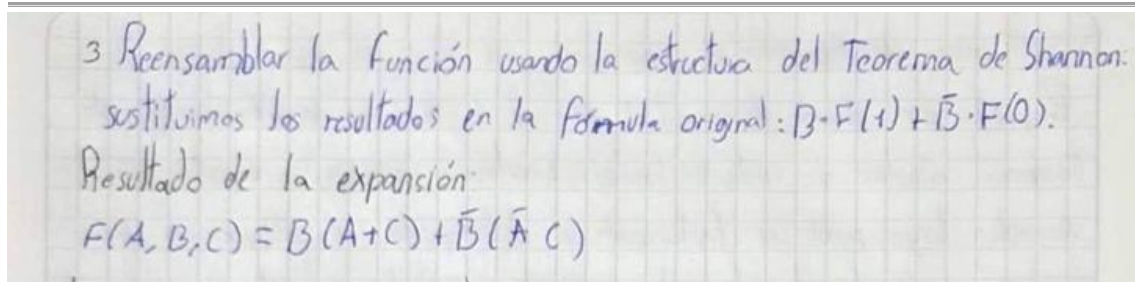
$$F(A, 1, C) = A \cdot (1) + \bar{A} \cdot C = A + \bar{A} \cdot C$$

Aplicando la regla de simplificación por absorción distributiva ( $A + \bar{A}C = A + C$ ):

$$F(A, 1, C) = A + C$$

2 Evaluación de la función cuando  $B=0$ :

$$F(A, 0, C) = A(0) + \bar{A} \cdot C = 0 + \bar{A}C = \bar{A} \cdot C$$



### Resultado de la Expansión

$$F(A, B, C) = B \cdot (A + C) + B' \cdot (A' \cdot C)$$

### Aplicación en la Vida Real y Conexión con Arquitectura de Computadores

En la ecuación resultante de la expansión, la variable B funciona exactamente como la línea de selección (Select) de un multiplexor de 2 a 1:

- Si  $B = 1$ , el multiplexor da paso a los datos de la subfunción  $(A + C)$ .
- Si  $B = 0$ , el multiplexor da paso a los datos de la subfunción  $(A' \cdot C)$ .

Esta propiedad es crítica para las FPGAs (Field Programmable Gate Arrays) y chips programables modernos, donde las funciones lógicas se implementan mediante LUTs (Look-Up Tables) basadas en el Teorema de Shannon.

## 5.2 FASE 5: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales)

### Ejercicio 1 - COMPUERTAS NAND

Ejercicio #1

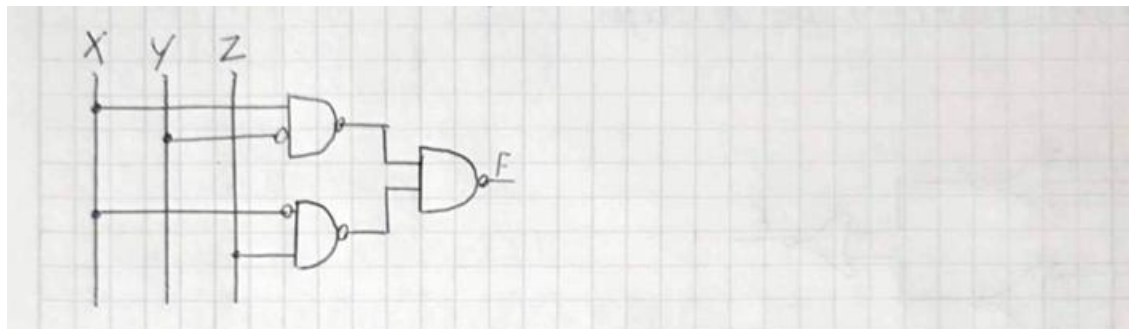
Función original  $\Rightarrow F = x\bar{y} + \bar{x}z$

$F = x\bar{y} + \bar{x}z$

$f = \overline{\overline{xy + \bar{x}z}}$  Doble negación

$F = (\overline{xy})(\overline{\bar{x}z})$  Ley de Morgan

#### Circuito



**Ejercicio 2 – COMPUERTAS NAND**

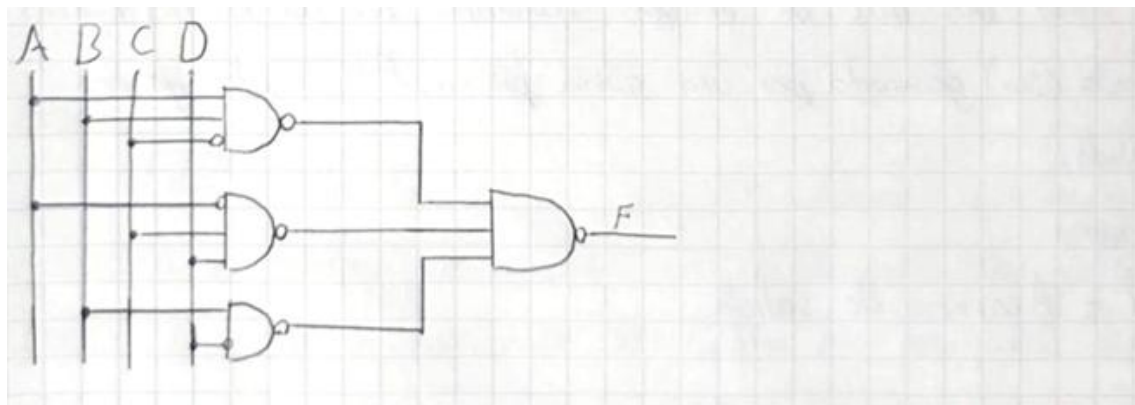
Ejercicio #2

Función original  $\Rightarrow F = AB\bar{C} + \bar{A}CD + B\bar{D}$

$F = AB\bar{C} + \bar{A}CD + B\bar{D}$

$F = \overline{AB\bar{C} + \bar{A}CD + B\bar{D}}$  Doble negación

$F = (\overline{AB\bar{C}})(\overline{\bar{A}CD})(\overline{B\bar{D}})$  Ley de Morgan

**Circuito**



### Ejercicio 3 – COMPUERTAS XOR

Ejercicio #3

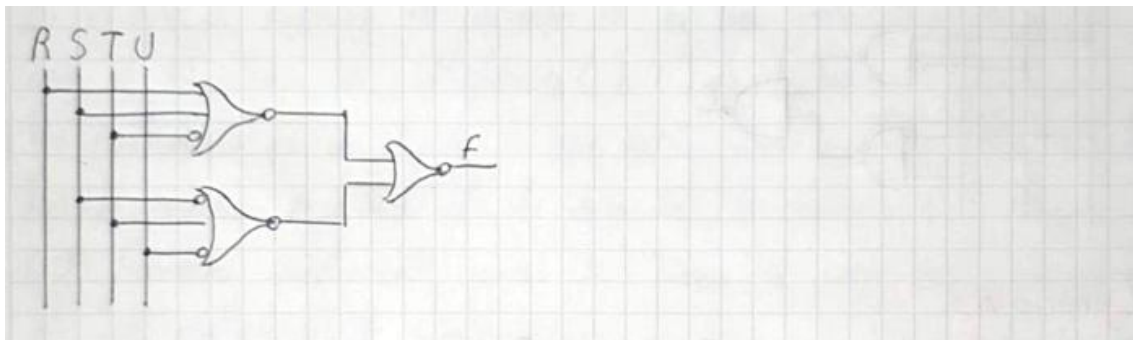
Función original:  $F = (R + S + \bar{T})(\bar{S} + T + \bar{U})$

$F = (R + S + \bar{T})(\bar{S} + T + \bar{U})$

$F = \overline{(R + S + T)(\bar{S} + T + \bar{U})}$  Doble negación

$F = \overline{(R + S + \bar{T}) + (\bar{S} + T + \bar{U})}$  Ley de Morgan

Circuito



### **5.3 FASE 5: Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales**

#### **Informe Reflexivo: Optimización Lógica en Sistemas Computacionales y su Impacto Ambiental**

##### **Introducción**

La optimización lógica es una técnica fundamental en el diseño de sistemas computacionales y circuitos digitales. Consiste en simplificar expresiones booleanas para reducir la cantidad de compuertas lógicas y componentes electrónicos necesarios para implementar una determinada función. Este proceso no solo mejora el rendimiento de los sistemas, sino que también contribuye a disminuir costos de fabricación, consumo energético y efectos ambientales asociados a la producción tecnológica.

##### **Desarrollo**

Los sistemas computacionales modernos dependen de millones de circuitos electrónicos que procesan información mediante operaciones lógicas. Cuando una expresión booleana se simplifica utilizando métodos como el Álgebra de Boole o los Mapas de Karnaugh, se reduce el número de compuertas requeridas para construir un circuito. Esta reducción implica un diseño más eficiente y menos complejo.

Desde el punto de vista económico, la optimización lógica permite disminuir la cantidad de materiales utilizados durante la fabricación de circuitos integrados. Al necesitar menos componentes electrónicos, los costos de producción, ensamblaje y mantenimiento también se reducen. Esto resulta especialmente importante en la industria tecnológica, donde pequeñas mejoras en el diseño pueden generar grandes ahorros cuando se fabrican miles o millones de dispositivos.

En cuanto al consumo energético, un circuito simplificado requiere menos transistores activos para realizar la misma función. Como consecuencia, se reduce la energía eléctrica consumida durante el funcionamiento del sistema. Esta característica es fundamental en dispositivos portátiles, centros de datos y equipos de procesamiento de alta demanda, donde la eficiencia energética representa un factor crítico para el rendimiento y la sostenibilidad.

Además, la optimización lógica tiene una relación directa con la reducción de la huella ambiental. La fabricación de componentes electrónicos requiere materias primas, energía y procesos industriales que generan emisiones contaminantes. Al disminuir la cantidad de elementos necesarios para construir un circuito, también se reduce el consumo de recursos naturales y la generación de residuos electrónicos. De esta manera, la optimización lógica contribuye indirectamente a la protección del medio ambiente y al desarrollo de tecnologías más sostenibles.

##### **Reflexión Crítica**

El análisis realizado permite comprender que la optimización lógica no debe considerarse únicamente como una herramienta matemática o técnica para simplificar circuitos. Su impacto trasciende el ámbito computacional y alcanza aspectos económicos, energéticos y ambientales. Como estudiante de Computación, considero que aprender técnicas de simplificación booleana es esencial, ya que permite desarrollar soluciones más eficientes y responsables.

En la actualidad, donde el uso de la tecnología crece constantemente, los profesionales de la informática y la electrónica tienen la responsabilidad de diseñar sistemas que aprovechen mejor los recursos disponibles. Cada compuerta eliminada mediante una



correcta optimización representa una oportunidad para reducir costos, ahorrar energía y disminuir el impacto ambiental de los dispositivos tecnológicos.

### **Conclusión**

La optimización lógica constituye una herramienta clave en el diseño de sistemas computacionales modernos. La simplificación de expresiones booleanas permite reducir la complejidad de los circuitos, disminuir costos de fabricación y mejorar la eficiencia energética de los dispositivos. Asimismo, contribuye a reducir la huella ambiental al requerir menos materiales y recursos durante los procesos de producción. Por estas razones, la optimización lógica no solo representa una mejora técnica, sino también una práctica alineada con los principios de sostenibilidad y responsabilidad tecnológica que deben caracterizar a los profesionales de la Computación.

**5.4 Práctica integradora: cada equipo resuelve un caso aplicado a la computación (diseño de un sumador binario de 1 bit, un comparador digital, o un codificador/decodificador) aplicando todas las técnicas de simplificación aprendidas.**

### Diseño de un sumador binario de 1 bit

PASO 5: Práctica integradora

- Diseño de un sumador binario de 1 bit.

1.- Introducción:

Un sumador completo de 1 bit es un circuito combinacional que permite sumar dos bits de entrada tomando en cuenta un acarreo de entrada ( $C_{in}$ ) generado por una suma previa. El circuito genera dos salidas.

- $S$  = suma
- $C_{out}$  = Acarreo de salida

### Tabla de Verdad

2.- Tabla de verdad

| A | B | $C_{in}$ | $S$ (suma) | $C_{out}$ (Acarreo) |
|---|---|----------|------------|---------------------|
| 0 | 0 | 0        | 0          | 0                   |
| 0 | 0 | 1        | 1          | 0                   |
| 0 | 1 | 0        | 1          | 0                   |
| 0 | 1 | 1        | 0          | 1                   |
| 1 | 0 | 0        | 1          | 0                   |
| 1 | 0 | 1        | 0          | 1                   |
| 1 | 1 | 0        | 0          | 1                   |
| 1 | 1 | 1        | 1          | 1                   |



## Función booleana

### 3.- Función booleana

- Función de la suma (S):

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

- Función del Acarreo (Cout):

$$Cout = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in}$$

## Simplificación con mapas de Karnaugh

### 4.- Simplificación mediante mapas de Karnaugh

| BC <sub>in</sub> | 00 | 01 | 11 | 10 |
|------------------|----|----|----|----|
| A                |    |    |    |    |
| 0                | 0  | 1  | 0  | 1  |
| 1                | 1  | 0  | 1  | 0  |

Análisis:

La distributiva de los "1" no permite formar agrupaciones que reduzcan el número de variables de la función. Por esta razón, la función de suma se representa de forma más completa mediante una operación XOR de tres variables

$$S = A \oplus B \oplus C_{in}$$

- Mapa de Karnaugh para Cout

| BC <sub>in</sub> | 00 | 01 | 11 | 10 |
|------------------|----|----|----|----|
| A                |    |    |    |    |
| 0                | 0  | 0  | 1  | 0  |
| 1                | 0  | 1  | 1  | 1  |

Agrupaciones

Grupo 1:  $m_3, m_7 \rightarrow BC_{in}$

Grupo 2:  $m_5, m_7 \rightarrow AC_{in}$

Grupo 3:  $m_6, m_7 \rightarrow AB$

Por lo tanto:

$$Cout = AB + AC_{in} + BC_{in}$$

## Algebra de Boole

### 5.- Simplificación por álgebra de Boole

#### • Función S

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

$$S = A(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in}) \quad \text{Agrupación}$$

Identificación de XOR:

$$\bar{B}C_{in} + B\bar{C}_{in} = B \oplus C_{in}$$

$$\bar{B}\bar{C}_{in} + BC_{in} = \overline{(B \oplus C_{in})}$$

Sustituyendo

$$S = A(B \oplus C_{in}) + A(\overline{B \oplus C_{in}})$$

Definición de XOR

$$S = A \oplus (B \oplus C_{in})$$

Asociativa

$$S = A \oplus B \oplus C_{in}$$

#### • Función Cout

$$C_{out} = \bar{A}B C_{in} + A\bar{B} C_{in} + A B \bar{C}_{in} + A B C_{in}$$

$$C_{out} = C_{in}(\bar{A}B + A\bar{B}) + AB(\bar{C}_{in} + C_{in}) \quad \text{Agrupación}$$

$$C_{out} = C_{in}(A \oplus B) + AB$$

Complemento y Definición  
de XOR

$$C_{out} = A \oplus B + AB C_{in}$$

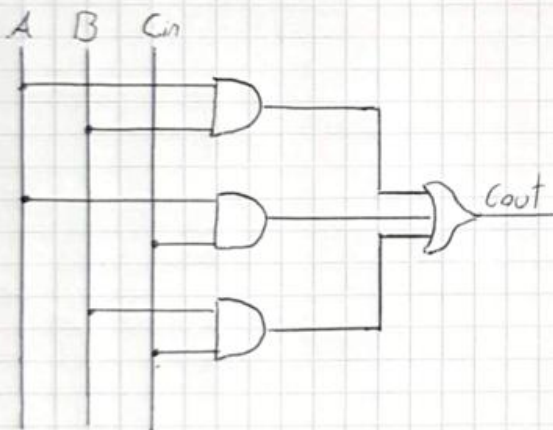
## Diseño del circuito

6.- Diseño de circuito

• Diseño 5: 2 compuertas XOR



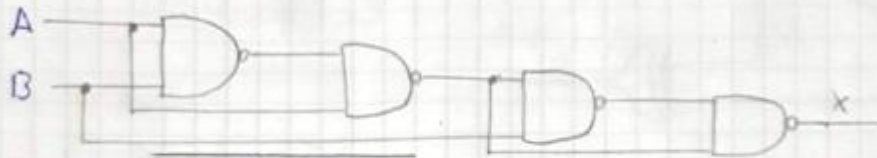
• Diseño Cout: AND de dos entradas y una compuerta OR de tres entradas



## 7. Implementación con compuertas NAND

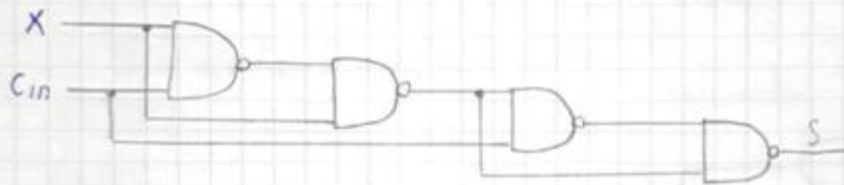
• Diseño  $S = A \oplus B \oplus C_{in}$

XOR1 :  $X = A \oplus B$



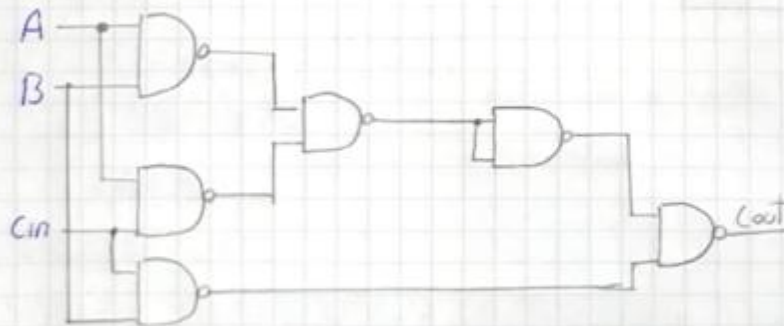
$$X = \overline{A \cdot A \cdot B} \cdot \overline{B \cdot A \cdot B}$$

XOR2 :  $S = X \oplus C_{in}$



$$S = \overline{X \cdot X \cdot C_{in}} \cdot \overline{C_{in} \cdot X \cdot C_{in}}$$

• Salida acarreo  $C_{out} = AB + AC_{in} + BC_{in}$



$$C_{out} = \overline{(\overline{A \cdot B \cdot A \cdot C_{in}}) \cdot B \cdot C_{in}}$$



## Jhoan Rojas

**5.1 FASE 5: Los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos.**

### **GUÍA DE INVESTIGACIÓN Y PRESENTACIÓN: LEYES Y PROPIEDADES ADICIONALES DEL ÁLGEBRA BOOLEANA**

#### **1. TEOREMA DEL CONSENSO DUAL (DUAL CONSENSUS THEOREM)**

##### **Definición y Explicación**

Mientras que el teorema del consenso estándar opera sobre una suma de productos, el Teorema del Consenso Dual se aplica directamente a expresiones en forma de producto de sumas (POS). Este teorema establece que en una combinación de tres variables lógicas sumadas en parejas, uno de los términos de la multiplicación es completamente redundante y puede ser eliminado sin alterar el comportamiento o resultado de la función.

##### **Identidad Matemática**

$$(A + B) \cdot (A' + C) \cdot (B + C) = (A + B) \cdot (A' + C)$$

Donde A' significa "A negado".

##### **Demostración intuitiva**

Analicemos el término  $(B + C)$ . Si tanto B como C son verdaderos (1), este paréntesis es verdadero. Sin embargo, debido a que la variable A debe tomar obligatoriamente un valor binario (ya sea 1 o 0), su presencia en los dos primeros términos  $((A + B) \text{ y } (A' + C))$  forzará a que uno de ellos dependa exclusivamente de los valores de B o C. Esto hace que el tercer paréntesis sea una repetición lógica que no aporta nueva información.

##### **Ejercicio Práctico Resuelto**

**Enunciado:** Simplificar la siguiente expresión lógica utilizada para el control de un circuito de automatización:

$$F = (X + Y) \cdot (X' + Z) \cdot (Y + Z)$$

## Solución Paso a Paso

Ejercicio práctico resuelto

Enunciado: Simplificar la siguiente expresión lógica utilizada para el control de un circuito de automatización:

$$F = (x + y)(\bar{x} + z)(y + z)$$

Solución paso a paso

- 1.- Identificamos las variables correspondientes a la identidad del teorema:
  - $A = x$
  - $B = y$
  - $C = z$
- 2.- Buscamos el término del consenso redundante, el cual está formado por las variables que acompañan a la variable que cambia de estado ( $x y \bar{x}$ ).
- 3.- Las variables acompañantes son  $y$  y  $z$ .
- 4.- El término redundante es  $(y + z)$ .
- 5.- Eliminando dicho término de la expresión.

Resultado simplificado:  $F = (x + y)(\bar{x} + z)$

### Resultado Simplificado:

$$F = (X + Y) \cdot (X' + Z)$$

### Aplicación en la Vida Real y Conexión con Diseño Digital

En el diseño de hardware y circuitos lógicos integrados, implementar la función original:

$$F = (X + Y) \cdot (X' + Z) \cdot (Y + Z)$$

requiere físicamente tres compuertas OR independientes que luego conectan sus salidas a una compuerta AND de tres entradas.

Al aplicar el Teorema del Consenso Dual, optimizamos el circuito reduciéndolo a sólo dos compuertas OR y una compuerta AND de dos entradas.



## 2. ÁLGEBRA BOOLEANA GENERAL: LEYES DE DE MORGAN

### Definición y Explicación

Las Leyes de De Morgan son herramientas fundamentales para la transformación de compuertas lógicas y la simplificación de condiciones complejas. Explican la equivalencia matemática que existe al distribuir o retirar una negación (operador NOT) de un paréntesis que contiene operaciones OR o AND.

### Identidades Matemáticas

#### Primera Ley:

$$(A \cdot B)' = A' + B'$$

(La multiplicación negada se convierte en suma de variables negadas)

#### Segunda Ley:

$$(A + B)' = A' \cdot B'$$

(La suma negada se convierte en multiplicación de variables negadas)

### Ejercicio Práctico Resuelto

**Enunciado:** Un sistema de desarrollo de software cuenta con la siguiente condición lógica anidada:

if not (status == "active" and points > 100):

## Solución Paso a Paso

2. Álgebra booleana general : Leyes de Morgan.

Ejercicio práctico resuelto

Enunciado : Un sistema de desarrollo de software cuenta con la siguiente condición lógica unificada:

if not (status == "active" and points > 100 ):

Solución paso a paso

Definimos las variables lógicas:

- $A = \text{status} == \text{"active"}$
- $B = \text{points} > 100$

La expresión actual del código es la negación de una multiplicación:

$$(A \cdot B)$$

Aplicamos la primera ley de Morgan:

$$(A \cdot B) = \overline{A + B}$$

Negamos cada variable individualmente:

- $\overline{A}$  significa que el status no es activo :  
 $\text{status} \neq \text{"active"}$
- $\overline{B}$  significa que los puntos son menores o iguales a 100  
 $\text{points} \leq 100$

Reemplazamos el operador AND por OR

Resultado simplificado en código:

if status != "active" or points <= 100 :





### **3. TEOREMA DE EXPANSIÓN DE SHANNON (SHANNON'S EXPANSION THEOREM)**

#### **Definición y Explicación**

Propuesto por Claude Shannon, este teorema establece que cualquier función booleana compleja puede ser descompuesta en subfunciones más pequeñas mediante el aislamiento de una variable de control.

#### **Identidad Matemática**

$$F(X_1, X_2, \dots, X_n) = X_1 \cdot F(1, X_2, \dots, X_n) + X_1' \cdot F(0, X_2, \dots, X_n)$$

#### **Ejercicio Práctico Resuelto**

**Enunciado:** Expandir la siguiente función lógica con respecto a la variable B:

$$F(A, B, C) = A \cdot B + A' \cdot C$$

#### **Solución Paso a Paso**

### 3- Teorema de expansión de Shannon

Ejercicio práctico resuelto

Enunciado: Expandir la siguiente función lógica con respecto a la variable B:

$$F(A, B, C) = AB + \bar{A}C$$

Solución paso a paso

1- Evaluar la condición cuando  $B=1$

$$F(A, 1, C) = A \cdot 1 + \bar{A} \cdot C$$

$$F(A, 1, C) = A + \bar{A}C$$

Aplicando la simplificación:

$$A + \bar{A}C = A + C$$

Resultado:

$$F(A, 1, C) = A + C$$

2- Evaluar cuando la función  $B=0$

$$F(A, 0, C) = A \cdot 0 + \bar{A} \cdot C$$

$$F(A, 0, C) = 0 + \bar{A} \cdot C$$

Resultado

$$F(A, 0, C) = \bar{A} \cdot C$$

3- Reensamblar la función

$$F(A, B, C) = B \cdot F(1) + \bar{B}(\bar{A} \cdot C)$$

Resultado:

$$F(A, B, C) = B(A + C) + \bar{B}(\bar{A} \cdot C)$$

### Resultado de la Expansión

$$F(A, B, C) = B \cdot (A + C) + B' \cdot (A' \cdot C)$$

### Aplicación en la Vida Real y Conexión con Arquitectura de Computadores

En la ecuación resultante de la expansión, la variable B funciona exactamente como la línea de selección (Select) de un multiplexor de 2 a 1:

- Si  $B = 1$ , el multiplexor da paso a los datos de la subfunción  $(A + C)$ .



- Si  $B = 0$ , el multiplexor da paso a los datos de la subfunción  $(A' \cdot C)$ .

Esta propiedad es crítica para las FPGAs (Field Programmable Gate Arrays) y chips programables modernos, donde las funciones lógicas se implementan mediante LUTs (Look-Up Tables) basadas en el Teorema de Shannon.

## 5.2 FASE 5: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales)

### Ejercicio 1 - COMPUERTAS NAND

Ejercicio 1

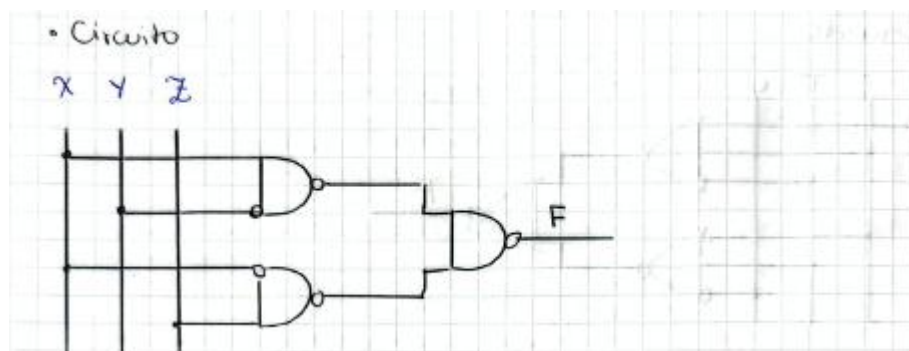
$$F = x\bar{y} + \bar{x}z$$
$$F = \overline{\overline{x\bar{y} + \bar{x}z}}$$

Doble negación

$$F = \overline{(x\bar{y})(\bar{x}z)}$$

Ley de Morgan

#### Circuito





## Ejercicio 2 – COMPUERTAS NAND

Ejercicio 2

$$F = ABC\bar{C} + \bar{A}CD + B\bar{D}$$

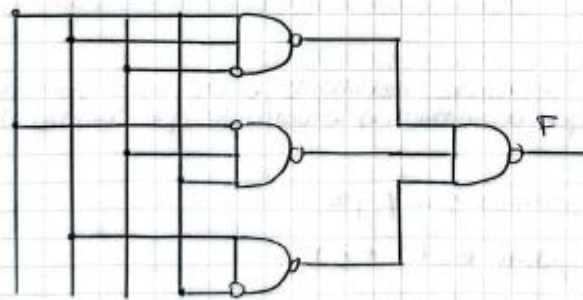
$$F = \overline{ABC\bar{C} + \bar{A}CD + B\bar{D}} \quad \text{Doble negación}$$

$$F = \overline{(ABC\bar{C})(\bar{A}CD)(B\bar{D})} \quad \text{Ley de Morgan}$$

### Circuito

• Circuito

X B C D

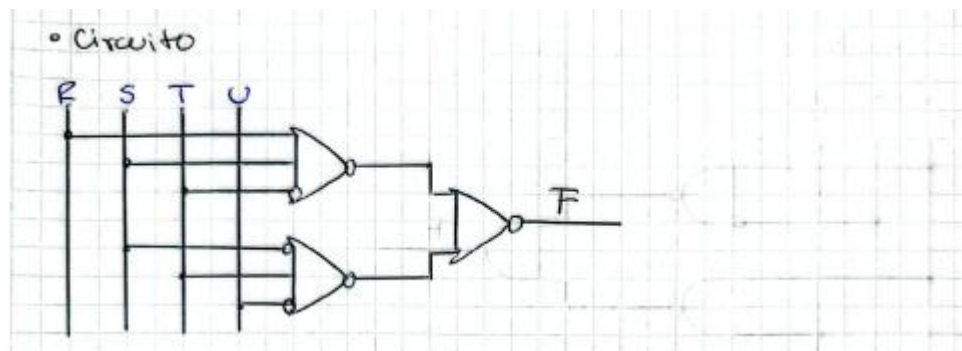


### Ejercicio 3 – COMPUERTAS XOR

Ejercicio 3

$$F = (R + S + \overline{T})(\overline{S} + T + \overline{U})$$
$$F = (R + S + \overline{T})(\overline{S} + T + \overline{U})$$
$$F = (R + S + \overline{T}) + (\overline{S} + T + \overline{U})$$

#### Circuito



### **5.3 FASE 5: Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales**

#### **Informe Reflexivo: Optimización Lógica en Sistemas Computacionales y su Impacto Ambiental**

##### **Introducción**

La optimización lógica es una técnica fundamental en el diseño de sistemas computacionales y circuitos digitales. Consiste en simplificar expresiones booleanas para reducir la cantidad de compuertas lógicas y componentes electrónicos necesarios para implementar una determinada función. Este proceso no solo mejora el rendimiento de los sistemas, sino que también contribuye a disminuir costos de fabricación, consumo energético y efectos ambientales asociados a la producción tecnológica.

##### **Desarrollo**

Los sistemas computacionales modernos dependen de millones de circuitos electrónicos que procesan información mediante operaciones lógicas. Cuando una expresión booleana se simplifica utilizando métodos como el Álgebra de Boole o los Mapas de Karnaugh, se reduce el número de compuertas requeridas para construir un circuito. Esta reducción implica un diseño más eficiente y menos complejo.

Desde el punto de vista económico, la optimización lógica permite disminuir la cantidad de materiales utilizados durante la fabricación de circuitos integrados. Al necesitar menos componentes electrónicos, los costos de producción, ensamblaje y mantenimiento también se reducen. Esto resulta especialmente importante en la industria tecnológica, donde pequeñas mejoras en el diseño pueden generar grandes ahorros cuando se fabrican miles o millones de dispositivos.

En cuanto al consumo energético, un circuito simplificado requiere menos transistores activos para realizar la misma función. Como consecuencia, se reduce la energía eléctrica consumida durante el funcionamiento del sistema. Esta característica es fundamental en dispositivos portátiles, centros de datos y equipos de procesamiento de alta demanda, donde la eficiencia energética representa un factor crítico para el rendimiento y la sostenibilidad.

Además, la optimización lógica tiene una relación directa con la reducción de la huella ambiental. La fabricación de componentes electrónicos requiere materias primas, energía y procesos industriales que generan emisiones contaminantes. Al disminuir la cantidad de elementos necesarios para construir un circuito, también se reduce el consumo de recursos naturales y la generación de residuos electrónicos. De esta manera, la optimización lógica contribuye indirectamente a la protección del medio ambiente y al desarrollo de tecnologías más sostenibles.

##### **Reflexión Crítica**

El análisis realizado permite comprender que la optimización lógica no debe considerarse únicamente como una herramienta matemática o técnica para simplificar circuitos. Su impacto trasciende el ámbito computacional y alcanza aspectos económicos, energéticos y ambientales. Como estudiante de Computación, considero que aprender técnicas de simplificación booleana es esencial, ya que permite desarrollar soluciones más eficientes y responsables.

En la actualidad, donde el uso de la tecnología crece constantemente, los profesionales de la informática y la electrónica tienen la responsabilidad de diseñar sistemas que aprovechen mejor los recursos disponibles. Cada compuerta eliminada mediante una



---

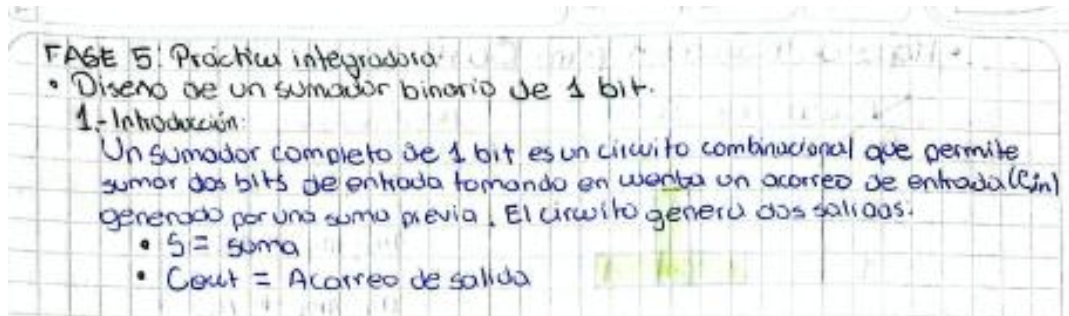
correcta optimización representa una oportunidad para reducir costos, ahorrar energía y disminuir el impacto ambiental de los dispositivos tecnológicos.

### **Conclusión**

La optimización lógica constituye una herramienta clave en el diseño de sistemas computacionales modernos. La simplificación de expresiones booleanas permite reducir la complejidad de los circuitos, disminuir costos de fabricación y mejorar la eficiencia energética de los dispositivos. Asimismo, contribuye a reducir la huella ambiental al requerir menos materiales y recursos durante los procesos de producción. Por estas razones, la optimización lógica no solo representa una mejora técnica, sino también una práctica alineada con los principios de sostenibilidad y responsabilidad tecnológica que deben caracterizar a los profesionales de la Computación.

**5.4 Práctica integradora: cada equipo resuelve un caso aplicado a la computación (diseño de un sumador binario de 1 bit, un comparador digital, o un codificador/decodificador) aplicando todas las técnicas de simplificación aprendidas.**

### Diseño de un sumador binario de 1 bit



### Tabla de Verdad

2.- Tabla de verdad

| A | B | $C_{in}$ | $S$ (suma) | $C_{out}$ (Acarreo) |
|---|---|----------|------------|---------------------|
| 0 | 0 | 0        | 0          | 0                   |
| 0 | 0 | 1        | 1          | 0                   |
| 0 | 1 | 0        | 1          | 0                   |
| 0 | 1 | 1        | 0          | 1                   |
| 1 | 0 | 0        | 1          | 0                   |
| 1 | 0 | 1        | 0          | 1                   |
| 1 | 1 | 0        | 0          | 1                   |
| 1 | 1 | 1        | 1          | 1                   |



## Función booleana

3. Función booleana

- Función de la suma ( $S$ ):

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

- Función del Acorreo ( $Cout$ ):

$$Cout = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in}$$

## Simplificación con mapas de Karnaugh

4. Simplificación mediante mapas de Karnaugh

- Mapa para  $S$

|     |           |    |    |    |    |
|-----|-----------|----|----|----|----|
|     | $BC_{in}$ | 00 | 01 | 11 | 10 |
| $A$ |           |    |    |    |    |
| 0   |           | 0  | 1  | 0  | 1  |
| 1   |           | 1  | 0  | 1  | 0  |

Análisis  
La distribución de los "1" no permite formar agrupaciones que reduzcan el número de variables de la función. Por esta razón, la función de suma se representa de forma más compacta mediante una operación XOR de tres variables.

$$S = A \oplus B \oplus C_{in}$$

- Mapa de Karnaugh para  $Cout$

|     |           |    |    |    |    |
|-----|-----------|----|----|----|----|
|     | $BC_{in}$ | 00 | 01 | 11 | 10 |
| $A$ |           |    |    |    |    |
| 0   |           | 0  | 0  | 1  | 0  |
| 1   |           | 0  | 1  | 1  | 1  |

Agrupaciones

- Grupo 1:  $m_3, m_7 \rightarrow BC_{in}$
- Grupo 2:  $m_3, m_7 \rightarrow AC_{in}$
- Grupo 3:  $m_6, m_7 \rightarrow AB$

Por lo tanto:

$$Cout = AB + AC_{in} + BC_{in}$$

## Algebra de Boole

## 5- Simplificación por álgebra de Boole

### • Función S

$$S = \overline{A} \overline{B} C_{in} + \overline{A} B \overline{C}_{in} + A \overline{B} \overline{C}_{in} + A B C_{in}$$

$$S = \overline{A} (\overline{B} C_{in} + B \overline{C}_{in}) + A (\overline{B} \overline{C}_{in} + B C_{in}) \quad \text{Agrupación}$$

Identificación de XOR:

$$\overline{B} C_{in} + B \overline{C}_{in} = B \oplus C_{in}$$

$$\overline{B} \overline{C}_{in} + B C_{in} = \overline{(B \oplus C_{in})}$$

Sustituyendo

$$S = \overline{A} (B \oplus C_{in}) + A (\overline{B \oplus C_{in}})$$

Definición de XOR

$$S = A \oplus (B \oplus C_{in})$$

Asociativa

$$S = A \oplus B \oplus C_{in}$$

### • Función Cout

$$C_{out} = \overline{A} B C_{in} + A \overline{B} C_{in} + A B \overline{C}_{in} + A B C_{in}$$

Agrupación

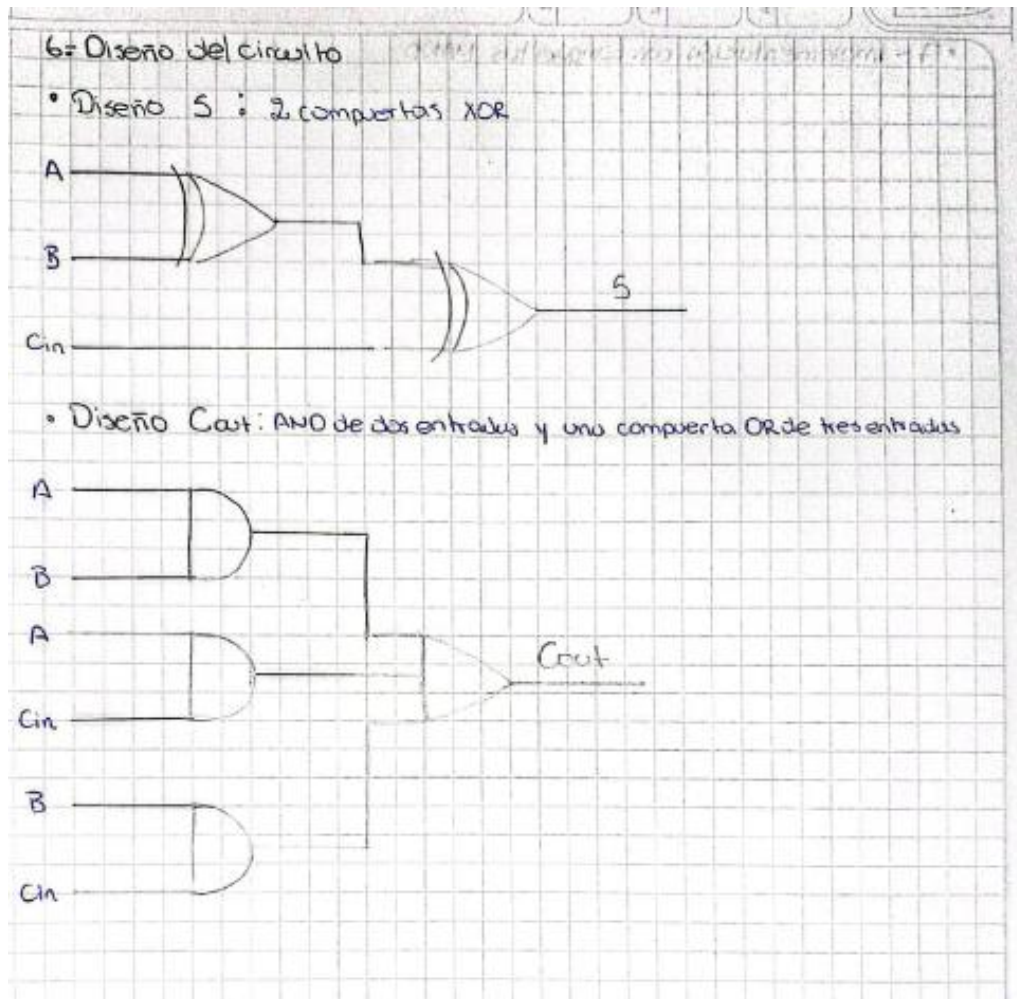
$$C_{out} = C_{in} (\overline{A} B + A \overline{B}) + A B \overline{C}_{in} + A B C_{in}$$

Complemento y Definición de XOR

$$C_{out} = C_{in} (A \oplus B) + A B$$

$$C_{out} = A B + A C_{in} + B C_{in}$$

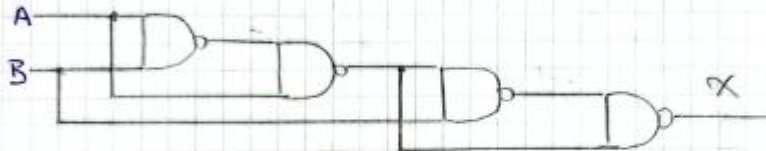
## Diseño del circuito



### 7. Implementación con compuertas NAND

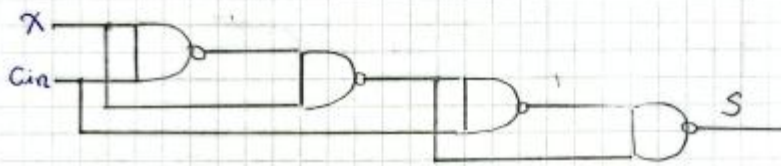
• Diseño  $S = A \oplus B \oplus C_{in}$

XOR 1 :  $X = A \oplus B$



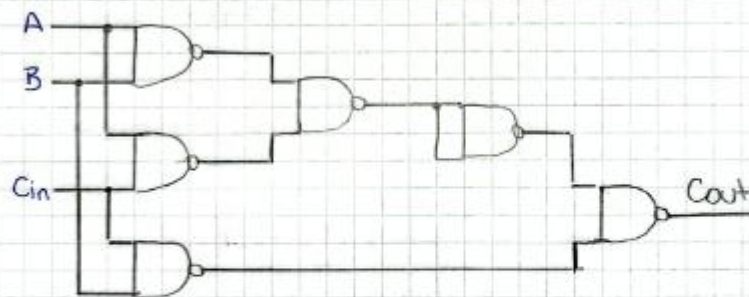
$$X = \overline{A \cdot A \cdot B} \cdot \overline{B \cdot A \cdot B}$$

XOR 2 :  $S = X \oplus C_{in}$



$$S = \overline{X \cdot X \cdot C_{in}} \cdot \overline{C_{in} \cdot X \cdot C_{in}}$$

• Salida acarreo  $C_{out} = AB + AC_{in} + BC_{in}$



$$C_{out} = \overline{(\overline{A \cdot B \cdot A \cdot C_{in}})} \cdot \overline{B \cdot C_{in}}$$

---

## Tyrone Encarnación

**5.1 FASE 5: Los estudiantes investigan y presentan de forma participativa las leyes y propiedades adicionales mediante ejemplos prácticos.**

### **GUÍA DE INVESTIGACIÓN Y PRESENTACIÓN: LEYES Y PROPIEDADES ADICIONALES DEL ÁLGEBRA BOOLEANA**

#### **1. TEOREMA DEL CONSENSO DUAL (DUAL CONSENSUS THEOREM)**

##### **Definición y Explicación**

Mientras que el teorema del consenso estándar opera sobre una suma de productos, el Teorema del Consenso Dual se aplica directamente a expresiones en forma de producto de sumas (POS). Este teorema establece que en una combinación de tres variables lógicas sumadas en parejas, uno de los términos de la multiplicación es completamente redundante y puede ser eliminado sin alterar el comportamiento o resultado de la función.

##### **Identidad Matemática**

$$(A + B) \cdot (A' + C) \cdot (B + C) = (A + B) \cdot (A' + C)$$

Donde  $A'$  significa "A negado".

##### **Demostración intuitiva**

Analicemos el término  $(B + C)$ . Si tanto B como C son verdaderos (1), este paréntesis es verdadero. Sin embargo, debido a que la variable A debe tomar obligatoriamente un valor binario (ya sea 1 o 0), su presencia en los dos primeros términos ( $(A + B)$  y  $(A' + C)$ ) forzará a que uno de ellos dependa exclusivamente de los valores de B o C. Esto hace que el tercer paréntesis sea una repetición lógica que no aporta nueva información.

##### **Ejercicio Práctico Resuelto**

**Enunciado:** Simplificar la siguiente expresión lógica utilizada para el control de un circuito de automatización:

$$F = (X + Y) \cdot (X' + Z) \cdot (Y + Z)$$



## Solución Paso a Paso

Ejercicio Práctico Resuelto

Simplificar la expresión lógica utilizada para el control de un circuito de automatización

$$F = (X + Y)(\bar{X} + Z)(Y + Z)$$

Solución Paso a Paso

1. Identificar variables correspondientes a la identidad del teorema:

$$A = X, B = Y, \text{ y } C = Z.$$

2. Buscar término de consenso redundante, el cual está formado por las variables que acompañan a la variable que cambia de estado ( $X$  y  $\bar{X}$ ). En este caso, las variables acompañantes son  $Y$  y  $Z$ .

3. El término de la expresión

4. Eliminamos el término de la expresión

Resultado simplificado:  $F = (X + Y)(\bar{X} + Z)$

### Resultado Simplificado:

$$F = (X + Y) \cdot (X' + Z)$$

### Aplicación en la Vida Real y Conexión con Diseño Digital

En el diseño de hardware y circuitos lógicos integrados, implementar la función original:

$$F = (X + Y) \cdot (X' + Z) \cdot (Y + Z)$$

requiere físicamente tres compuertas OR independientes que luego conectan sus salidas a una compuerta AND de tres entradas.

Al aplicar el Teorema del Consenso Dual, optimizamos el circuito reduciéndolo a sólo dos compuertas OR y una compuerta AND de dos entradas.



## 2. ÁLGEBRA BOOLEANA GENERAL: LEYES DE DE MORGAN

### Definición y Explicación

Las Leyes de De Morgan son herramientas fundamentales para la transformación de compuertas lógicas y la simplificación de condiciones complejas. Explican la equivalencia matemática que existe al distribuir o retirar una negación (operador NOT) de un paréntesis que contiene operaciones OR o AND.

### Identidades Matemáticas

#### Primera Ley:

$$(A \cdot B)' = A' + B'$$

(La multiplicación negada se convierte en suma de variables negadas)

#### Segunda Ley:

$$(A + B)' = A' \cdot B'$$

(La suma negada se convierte en multiplicación de variables negadas)

### Ejercicio Práctico Resuelto

**Enunciado:** Un sistema de desarrollo de software cuenta con la siguiente condición lógica anidada:

if not (status == "active" and points > 100):

## Solución Paso a Paso

Ejercicio práctico resuelto

Solución paso a paso:

1 Definir variables lógicas:

-  $A = \text{status} == \text{"active"}$

-  $B = \text{points} > 100$

2 La expresión actual del código es la negación de una multiplicación  
 $\overline{A \cdot B}$

3 Aplicamos la primera ley de Morgan:  $\overline{A \cdot B} = \overline{A} + \overline{B}$

4 Negamos cada variable individualmente.

-  $\overline{A}$  significa que el status no es activo. ( $\text{status} \neq \text{"active"}$ )

-  $\overline{B}$  significa que los puntos no son mayores a 100, es decir, son menores o iguales ( $\text{points} \leq 100$ )

5 Reemplazamos el operador and por un or.

Resultado simplificado en código

```
if status != "active" or points <= 100
```



---

### 3. TEOREMA DE EXPANSIÓN DE SHANNON (SHANNON'S EXPANSION THEOREM)

#### Definición y Explicación

Propuesto por Claude Shannon, este teorema establece que cualquier función booleana compleja puede ser descompuesta en subfunciones más pequeñas mediante el aislamiento de una variable de control.

#### Identidad Matemática

$$F(X_1, X_2, \dots, X_n) = X_1 \cdot F(1, X_2, \dots, X_n) + X_1' \cdot F(0, X_2, \dots, X_n)$$

#### Ejercicio Práctico Resuelto

**Enunciado:** Expandir la siguiente función lógica con respecto a la variable B:

$$F(A, B, C) = A \cdot B + A' \cdot C$$

#### Solución Paso a Paso



Ejercicio práctico resuelto

Expandir la siguiente función lógica con respecto a la variable B:

$$F(A, B, C) = AB + \bar{A}C$$

Solución paso a paso.

1. Evaluar la función cuando  $B=1$ :

$$F(A, 1, C) = A \cdot (1) + \bar{A} \cdot C = A + \bar{A} \cdot C$$

Aplicando regla de simplificación por absorción distributiva

$$(A + \bar{A}C = A + C) =$$

$$F(A, 1, C) = A + C$$

Evaluación de la función cuando  $B=0$

$$F(A, 0, C) = A(0) + \bar{A} \cdot C = 0 + \bar{A}C = \bar{A} \cdot C$$

Reensamblar la función cuando la estructura del Teorema de Shannon: sustituimos los resultados en la fórmula original

$$B \cdot F(1) + \bar{B} \cdot F(0)$$

Resultado de la expansión.

$$F(A, B, C) = B(A + C) + \bar{B}(\bar{A}C)$$

### Resultado de la Expansión

$$F(A, B, C) = B \cdot (A + C) + B' \cdot (A' \cdot C)$$

### Aplicación en la Vida Real y Conexión con Arquitectura de Computadores

En la ecuación resultante de la expansión, la variable B funciona exactamente como la línea de selección (Select) de un multiplexor de 2 a 1:

- Si  $B = 1$ , el multiplexor da paso a los datos de la subfunción  $(A + C)$ .
- Si  $B = 0$ , el multiplexor da paso a los datos de la subfunción  $(A' \cdot C)$ .





---

Esta propiedad es crítica para las FPGAs (Field Programmable Gate Arrays) y chips programables modernos, donde las funciones lógicas se implementan mediante LUTs (Look-Up Tables) basadas en el Teorema de Shannon.

## 5.2 FASE 5: Práctica de implementación de funciones booleanas usando únicamente compuertas NAND o NOR (puertas universales)

### Ejercicio 1 - COMPUERTAS NAND

Ejercicio #1

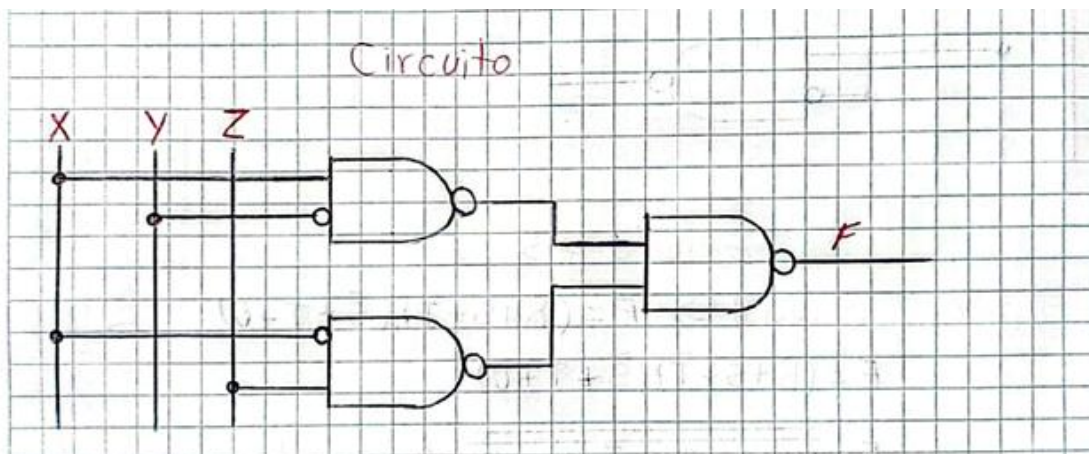
Función original  $\Rightarrow F = x\bar{y} + \bar{x}z$

$F = x\bar{y} + \bar{x}z$

$F = \overline{\overline{x\bar{y} + \bar{x}z}}$  Ley de doble negación

$F = \overline{(x\bar{y}) \cdot (\bar{x}z)}$  Ley de morgan

### Circuito



## Ejercicio 2 – COMPUERTAS NAND

Ejercicio #2

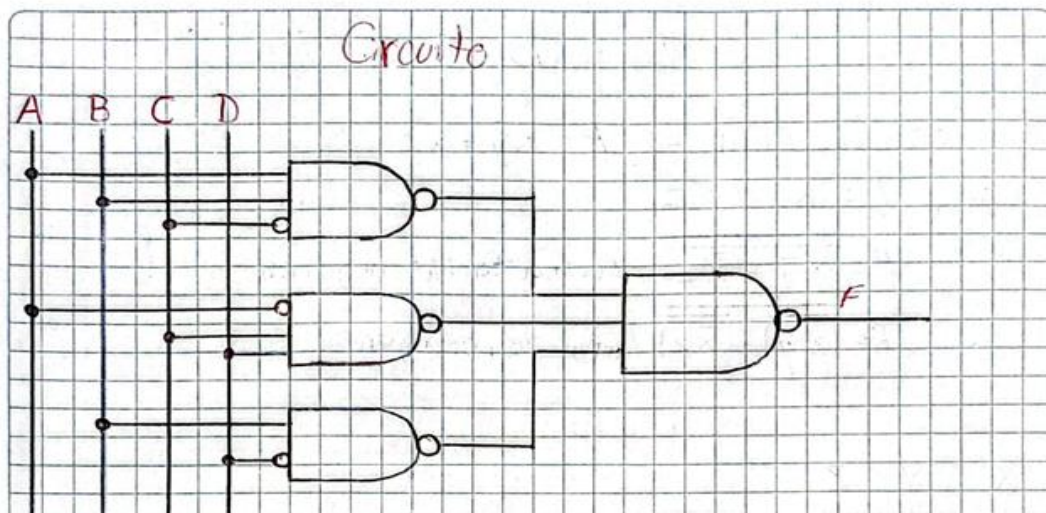
Función original  $\Rightarrow F = ABC\bar{C} + \bar{A}CD + B\bar{D}$

$F = ABC\bar{C} + \bar{A}CD + B\bar{D}$

$F = \overline{ABC\bar{C} + \bar{A}CD + B\bar{D}}$  Ley de doble negación

$F = \overline{(ABC\bar{C}) \cdot (\bar{A}CD) \cdot (B\bar{D})}$  Ley de Morgan

### Circuito



### Ejercicio 3 – COMPUERTAS XOR

Ejercicio #3

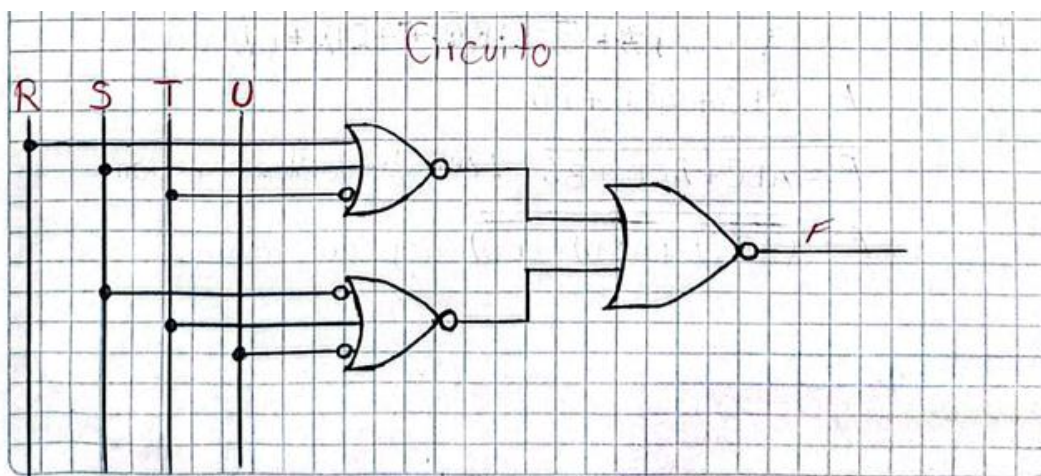
Función original  $\Rightarrow F = (R+S+\bar{T})(\bar{S}+T+\bar{U})$

$F = (R+S+\bar{T})(\bar{S}+T+\bar{U})$

$F = \overline{\overline{(R+S+\bar{T})(\bar{S}+T+\bar{U})}}$  Ley de doble negación

$F = \overline{(R+S+\bar{T}) + (\bar{S}+T+\bar{U})}$  Ley de Morgan

#### Circuito



### **5.3 FASE 5: Los equipos elaboran un informe reflexivo sobre la optimización lógica en sistemas computacionales**

#### **Informe Reflexivo: Optimización Lógica en Sistemas Computacionales y su Impacto Ambiental**

##### **Introducción**

La optimización lógica es una técnica fundamental en el diseño de sistemas computacionales y circuitos digitales. Consiste en simplificar expresiones booleanas para reducir la cantidad de compuertas lógicas y componentes electrónicos necesarios para implementar una determinada función. Este proceso no solo mejora el rendimiento de los sistemas, sino que también contribuye a disminuir costos de fabricación, consumo energético y efectos ambientales asociados a la producción tecnológica.

##### **Desarrollo**

Los sistemas computacionales modernos dependen de millones de circuitos electrónicos que procesan información mediante operaciones lógicas. Cuando una expresión booleana se simplifica utilizando métodos como el Álgebra de Boole o los Mapas de Karnaugh, se reduce el número de compuertas requeridas para construir un circuito. Esta reducción implica un diseño más eficiente y menos complejo.

Desde el punto de vista económico, la optimización lógica permite disminuir la cantidad de materiales utilizados durante la fabricación de circuitos integrados. Al necesitar menos componentes electrónicos, los costos de producción, ensamblaje y mantenimiento también se reducen. Esto resulta especialmente importante en la industria tecnológica, donde pequeñas mejoras en el diseño pueden generar grandes ahorros cuando se fabrican miles o millones de dispositivos.

En cuanto al consumo energético, un circuito simplificado requiere menos transistores activos para realizar la misma función. Como consecuencia, se reduce la energía eléctrica consumida durante el funcionamiento del sistema. Esta característica es fundamental en dispositivos portátiles, centros de datos y equipos de procesamiento de alta demanda, donde la eficiencia energética representa un factor crítico para el rendimiento y la sostenibilidad.

Además, la optimización lógica tiene una relación directa con la reducción de la huella ambiental. La fabricación de componentes electrónicos requiere materias primas, energía y procesos industriales que generan emisiones contaminantes. Al disminuir la cantidad de elementos necesarios para construir un circuito, también se reduce el consumo de recursos naturales y la generación de residuos electrónicos. De esta manera, la optimización lógica contribuye indirectamente a la protección del medio ambiente y al desarrollo de tecnologías más sostenibles.

##### **Reflexión Crítica**

El análisis realizado permite comprender que la optimización lógica no debe considerarse únicamente como una herramienta matemática o técnica para simplificar circuitos. Su impacto trasciende el ámbito computacional y alcanza aspectos económicos, energéticos y ambientales. Como estudiante de Computación, considero que aprender técnicas de simplificación booleana es esencial, ya que permite desarrollar soluciones más eficientes y responsables.

En la actualidad, donde el uso de la tecnología crece constantemente, los profesionales de la informática y la electrónica tienen la responsabilidad de diseñar sistemas que aprovechen mejor los recursos disponibles. Cada compuerta eliminada mediante una





correcta optimización representa una oportunidad para reducir costos, ahorrar energía y disminuir el impacto ambiental de los dispositivos tecnológicos.

### **Conclusión**

La optimización lógica constituye una herramienta clave en el diseño de sistemas computacionales modernos. La simplificación de expresiones booleanas permite reducir la complejidad de los circuitos, disminuir costos de fabricación y mejorar la eficiencia energética de los dispositivos. Asimismo, contribuye a reducir la huella ambiental al requerir menos materiales y recursos durante los procesos de producción. Por estas razones, la optimización lógica no solo representa una mejora técnica, sino también una práctica alineada con los principios de sostenibilidad y responsabilidad tecnológica que deben caracterizar a los profesionales de la Computación.

**5.4 Práctica integradora:** cada equipo resuelve un caso aplicado a la computación (diseño de un sumador binario de 1 bit, un comparador digital, o un codificador/decodificador) aplicando todas las técnicas de simplificación aprendidas.

### Diseño de un sumador binario de 1 bit

**Diseño de un sumador binario de 1 bit.**

**1. Introducción**

Un sumador completo de 1 bit es un circuito combinatorial que permite sumar dos bits de entrada tomando en cuenta un acarreo de entrada ( $C_{in}$ ) generado por una suma previa. El circuito genera dos salidas:

- $S$  = Suma
- $C_{out}$  = Acarreo de salida

### Tabla de Verdad

**2. Tabla de verdad**

| A | B | $C_{in}$ | S (Suma) | $C_{out}$ (Acarreo) |
|---|---|----------|----------|---------------------|
| 0 | 0 | 0        | 0        | 0                   |
| 0 | 0 | 1        | 1        | 0                   |
| 0 | 1 | 0        | 1        | 0                   |
| 0 | 1 | 1        | 0        | 1                   |
| 1 | 0 | 0        | 1        | 0                   |
| 1 | 0 | 1        | 0        | 1                   |
| 1 | 1 | 0        | 0        | 1                   |
| 1 | 1 | 1        | 1        | 1                   |

## Función booleana

### 3. Función booleana

• Función de la suma (s):

$$S = \bar{A}B\bar{C}_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

Función del Acarreo (cout):

$$C_{out} = \bar{A}B\bar{C}_{in} + \bar{A}BC_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

## Simplificación con mapas de Karnaugh

### 4. Simplificación mediante Mapas de Karnaugh

• Mapa para S

| BC <sub>in</sub> | 00 | 01 | 11 | 10 |
|------------------|----|----|----|----|
| A                |    |    |    |    |
| 0                | 0  | 1  | 0  | 1  |
| 1                | 1  | 0  | 1  | 0  |

Análisis:

La distribución de los "1" no permite formar agrupaciones que reduzcan el número de variables de la función. Por esta razón, la función de suma se representa de forma más compacta mediante una operación xor de tres variables.

$$S = A \oplus B \oplus C_{in}$$

• Mapa de Karnaugh para Cout

| BC <sub>in</sub> | 00 | 01 | 11 | 10 |
|------------------|----|----|----|----|
| A                |    |    |    |    |
| 0                | 0  | 0  | 1  | 0  |
| 1                | 0  | 1  | 1  | 1  |

Agrupaciones

Grupo 1:  
 $m_5, m_7 \rightarrow B\bar{C}_{in}$

Grupo 2:  
 $m_5, m_7 \rightarrow AC_{in}$

Grupo 3:  
 $m_6, m_7 \rightarrow AB$

Por lo tanto:

$$C_{out} = AB + AC_{in} + B\bar{C}_{in}$$



## Álgebra de Boole

### Simplificación por Álgebra de Boole

• Función S

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

$$S = \bar{A}(BC_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in})$$

Agrupación

Identificación de XOR:

$$\bar{B}C_{in} + B\bar{C}_{in} = B \oplus C_{in}$$

$$\bar{B}\bar{C}_{in} + BC_{in} = \overline{(B \oplus C_{in})}$$

Sustituyendo

$$S = \bar{A}(B \oplus C_{in}) + A(\overline{(B \oplus C_{in})})$$

Definición de XOR

$$S = A \oplus (B \oplus C_{in})$$

$$S = A \oplus B \oplus C_{in}$$

Asociativa

### Función Cout

$$C_{out} = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

Agrupación

$$C_{out} = C_{in}(\bar{A}\bar{B} + A\bar{B}) + AB(\bar{C}_{in} + C_{in})$$

$$C_{out} = C_{in}(A \oplus B) + AB$$

Complemento y Definición de XOR

$$C_{out} = AB + AC_{in} + BC_{in}$$



Universidad  
Nacional  
de Loja

FEIRNNR - Carrera de Computación

## **Diseño del circuito**

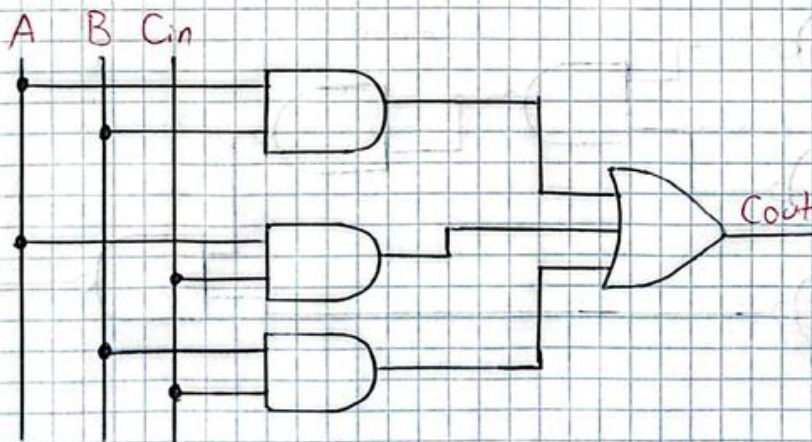


## 6. Diseño del circuito

- Diseño S: 2 compuertas XOR



- Diseño Cout: AND de dos entradas y una compuerta OR de tres entradas.

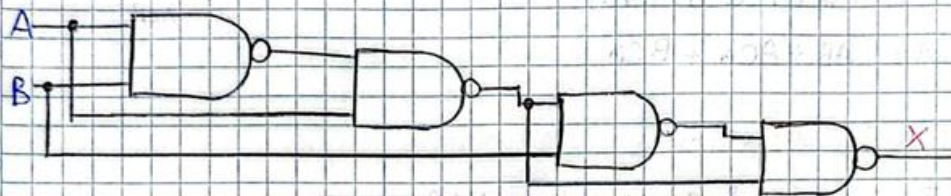


|     |     |     |      |
|-----|-----|-----|------|
| DIA | MES | AÑO | NOTA |
|     |     |     | /    |

### 7. Implementación con compuertas NAND

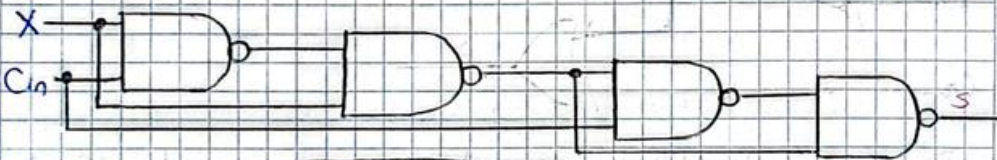
- Diseño  $S = A \oplus B \oplus C_{in}$

XOR 1:  $X = A \oplus B$



$$X = A \cdot \overline{A \cdot B} \cdot \overline{B \cdot A} \cdot \overline{A \cdot B}$$

XOR 2:  $S = X \oplus C_{in}$



$$S = X \cdot \overline{X \cdot C_{in}} \cdot \overline{C_{in} \cdot X} \cdot \overline{X \cdot C_{in}}$$

